

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 面向对象程序设计基础

实验时间 2025 年 9 月 12、19、26 日第 3、4 节

年级 2024 专业班级 计算机科学与技术1班

姓名 网球拍 学 号 20050090

评分：

教师签名： 王庆平

上机一 面向对象程序设计基础

一、上机目的

- 1、熟悉类的封装特性，能够说出为什么要封装以及如何实现封装。
- 2、掌握构造方法的定义和重载，能够独立定义构造方法，重载构造方法。
- 3、熟悉 this 关键字，能够使用 this 调用成员属性、成员方法、构造方法。
- 4、熟悉 static 关键字的使用，能够说出静态（属性、方法）的特点。
- 5、了解面向对象中的继承特性，能够说出继承的概念与特点。
- 6、掌握方法的重写，能够在子类中重写父类方法。
- 7、掌握 super 关键字，能够在类中使用 super 关键字访问父类成员。
- 8、掌握 final 关键字，能够灵活使用 final 关键字修饰类、方法和变量。
- 9、掌握抽象类，能够熟练实现抽象类的定义与使用。
- 10、掌握接口，能够独立进行接口的编写。
- 11、掌握多态，能够熟练使用对象类型转换解决继承中的多态问题。
- 12、熟悉内部类，能够说出 4 种内部类的特点。

二、上机任务

1. 编写程序，演示如何实现类的封装。定义一个 Student 类，对 age 这属性进行封装，然后测试。

程序：

```
package com.exa01;

/**
 * 什么是封装？
 * 封装就是将对象中的实现细节隐藏。不被外界所直接访问。
 * 封装的好处？
 * 可以提高代码的安全性
 */
public class Student {
    private int age; //封装—私有化，用 private 修饰
    String name;
    String num;
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        if(age<=0||age>=30)
            System.out.println("年龄输入有误!!!");
        else
            this.age = age;
    }
    public void study() {
```

```

        System.out.println("正在努力地学习!!!");
    }
    public void play() {
        System.out.println("正在玩游戏!!!");
    }
}
%-----%
public class Test01 {
    public static void main(String[] args) {
        Student stu1=new Student();
        stu1.setAge(18);
        stu1.name="wqp";
        stu1.num="20050090";
        stu1.study();
        stu1.play();
        System.out.println("姓名: "+stu1.name+" 学号: "+stu1.num+" 年龄: "+stu1.getAge());
    }
}

```

运行结果:

```

"C:\Program Files\Java\jdk1.8.0
正在努力地学习!!!
正在玩游戏!!!
姓名: wqp 学号: 20050090 年龄: 18

```

图 1.1 运行结果示意图

2. 编写程序，演示构造方法的定义,演示构造方法的重载，定义一个 Person 类，然后测试。

程序:

```

package exa02;
/**
 * 构造方法的定义格式:
 * 权限修饰符 方法名 () {
 *     方法体;
 * }
 * 1. 方法名和类名要保持一致
 * 2. 构造方法没有返回值类型，连 void 都不能写
 * 3. 构造方法中不能写 return 语句
 * 构造方法的注意事项:
 * 1. 没有写任何的构造方法。系统会默认提供一个无参的构造方法。
 * 2. 如果自己写了一个有参的构造方法，那么系统就不会提供空参的构造方法了。
 * 构造方法重载:
 * 1. 只要每个构造方法的参数类型或参数个数不同即可。
 */
public class Person {

```

```

String name;
int age;
public Person() {
}
public Person(String name) {
    this.name = name;
}
public Person(int age) {
    this.age = age;
}
public Person(String name, int age) {
    this.name = name;
    this.age = age;
}
public void speak() {
    System.out.println("我叫: "+name+" ,今年"+age+"岁了! ");
}
}
%-----%
package exa02;
public class Test02 {
    public static void main(String[] args) {
        Person p1 =new Person();
        Person p2 =new Person(18);
        Person p3 =new Person("wqp");
        Person p4 =new Person("wqp",18);
        p4.speak();
    }
}

```

运行结果:



```

"C:\Program Files\Ja
我叫: wqp ,今年18岁了!

```

图 1.2 运行结果示意图

3. 编写程序，演示 this 关键字的使用，定义一个 Animal 类，然后测试。

程序:

```

package com.company.wqp03;
/**
 * this 关键字:
 * 1. 可以区分局部变量和成员变量同名的问题
 * 2. 代表的是当前对象的引用。谁来调用我，我就代表谁。
 * 3. 在构造方法中可以调用本类中其他构造方法 this（参数），必须放在第一行
 * 4. 在成员方法中调用其他成员方法 this.方法名（），this 可以省掉不写。
 */

```

```

public class Animal {
    int age;
    String name;
    public Animal() {
        this(3,"小黄");//调用有参的构造方法
    }
    public Animal(int age, String name) {
        this.age = age;
        this.name = name;
    }
    public void shout() {
        System.out.println("动物会叫!!!");
    }
    public void run() {
        System.out.println("动物会跑!!!");
    }
    public void show() {
        this.shout();
        run();//this 可以省掉不写
    }
}

%-----%

package com.company.wqp03;
public class Test03 {
    public static void main(String[] args) {
        Animal an1=new Animal(2,"小白");
        System.out.println("我叫: "+an1.name+" ,今年"+an1.age+"岁了!!");
        an1.show();
        Animal an2=new Animal();
        System.out.println("我叫: "+an2.name+" ,今年"+an2.age+"岁了!!");
    }
}

```

运行结果:

```

"C:\Program Files\Java
我叫: 小白 ,今年2岁了!!
动物会叫!!!
动物会跑!!!
我叫: 小黄 ,今年3岁了!!

```

图 1.3 运行结果示意图

4. 编写程序，演示 static 关键字的使用（修饰成员变量）。

程序:

```

package com.computer1.exa04;
/*

```

```

*   static 关键字：
*   修饰变量，变量可以被所有的对象所共享。
*   什么时候将变量定义成静态的呢？
*   1. 当一个变量需要被类中的所有对象所共享的时候。
*   2. 可以通过 类名.静态变量 访问静态变量。
*   3. 只能修饰成员变量。
*/
public class Student {
    int age;
    String name;
    static String School;
    public Student(int age, String name, String school) {
        this.age = age;
        this.name = name;
        School = school;
    }
    public void study() {
        System.out.println("学生会学习!!!");
    }
    public void play() {
        System.out.println("学生会玩游戏!!!");
    }

    @Override
    public String toString() {
        return "age=" + age + ", name=" + name + ", School=" + School;
    }
}

%-----%

package com.computer1.exa04;
public class Test04 {
    public static void main(String[] args) {
        Student stu1=new Student(20,"李四","滇池学院");
        Student stu2=new Student(19,"张三","滇池学院");
        Student stu3=new Student(18,"王五","滇池学院");
        System.out.println("%修改之前-----%");
        System.out.println(stu1.toString());
        System.out.println(stu2);
        System.out.println(stu3);
        System.out.println("%修改之后-----%");
        Student.School="滇池大学";
        System.out.println(stu1.toString());
        System.out.println(stu2);
        System.out.println(stu3);
    }
}

```

```

    }
}

```

运行结果：

```

"C:\Program Files\Java\jdk1.8.0_18
%修改之前-----%
age=20, name=李四, School=滇池学院
age=19, name=张三, School=滇池学院
age=18, name=王五, School=滇池学院
%修改之后-----%
age=20, name=李四, School=滇池大学
age=19, name=张三, School=滇池大学
age=18, name=王五, School=滇池大学

```

图 1.4 运行结果示意图

5. 编写程序，演示 static 关键字的使用（修饰方法）。

程序：

```

package com.computer1.exa05;

/*
 * 静态方法可以直接通过类名来调用。
 * 静态属于类。随着类的加载而加载。
 * 非静态属于对象。随着对象的创建而加载。
 * 静态和非静态的访问特点：
 * 1. 静态的只能访问静态的
 * 2. 非静态的既可以访问静态的。也可以访问非静态的。
 * 3. 静态方法多在工具类中。
 */

public class Animal {
    public static void method() {
        System.out.println("这是一个静态方法!!!");
    }

    public void show() {
        method();
        System.out.println("这是一个非静态方法!!!");
    }
}

%-----%

package com.computer1.exa05;
public class Test05 {
    public static void main(String[] args) {
        Animal an1=new Animal();
        an1.show();
        Animal.method();
    }
}

```

```

    }
}

```

运行结果：

```

"C:\Program Files\J
这是一个静态方法!!!
这是一个非静态方法!!!
这是一个静态方法!!!

```

图 1.5 运行结果示意图

6、编写程序，学习子类是如何继承父类的。

程序：

```

package com.company.wqp06;

/**
 * 继承：
 * 1. 什么是继承？
 * 说白了就是让类与类之间产生了关系。子父类的关系
 * 2. 继承的关键字。
 * 子类 extends 父类
 * 3. java 中的继承的特点。
 * 只支持单继承。但是可以多层继承。
 * 4. 继承父类只能使用父类的公共成员。
 * 5. 继承的好处和弊端。
 * 好处：提高了代码的复用性。提高代码的维护性。
 * 弊端：类与类之间的耦合性太强。
 */
public class Animal {
    private int age;
    private String color;
    private String name;
    public Animal() {
    }

    public Animal(int age, String color, String name) {
        this.age = age;
        this.color = color;
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String getColor() {

```



```

        return color;
    }
    public void setColor(String color) {
        this.color = color;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void shout() {
        System.out.println("动物会叫!!!");
    }
    public void eat() {
        System.out.println("动物会吃饭!!!");
    }
}
%-----%

package com.company.wqp06;
public class Dog extends Animal {
    // 什么都没有定义，其实都有了。
}
%-----%

package com.company.wqp06;
public class Cat extends Animal{
    // 什么都没有定义，其实都有了。
}
%-----%

package com.company.wqp06;
public class Test06 {
    public static void main(String[] args) {
        Cat cat1=new Cat();
        cat1.setAge(2);
        cat1.setColor("黄色");
        cat1.setName("波斯猫");
        System.out.println("名字: "+cat1.getName()+"，年龄: "+cat1.getAge()
+ "，颜色: "+cat1.getColor());
        Dog dog1=new Dog();
        dog1.eat();
        dog1.shout();
    }
}

```

运行结果:

```
"C:\Program Files\Java\jdk-1
名字：波斯猫，年龄：2，颜色：黄色
动物会吃饭!!!
动物会叫!!!
```

图 1.6 运行结果示意图

7、编写程序，演示方法的重写。

程序：

```
package com.company.wqp07;

/**
 * 方法的重写--为什么要进行方法的重写？
 * 方法的重写发生在子父类关系中。
 * 当父类提供的功能不满足子类具体的需求的时候，那么就需要对父类的方法进行重写。
 */
public class Animal {
    private int age;
    private String name;
    private String color;
    public Animal() {
    }
    public void shout() {
        System.out.println("动物会叫!!!");
    }
    public void eat() {
        System.out.println("动物会吃饭!!!");
    }
}

%-----%

package com.company.wqp07;
public class Cat extends Animal{
    @Override
    public void shout() {
        System.out.println("猫叫：喵喵!!!");
    }
    @Override
    public void eat() {
        System.out.println("猫吃鱼!!!");
    }
}

%-----%

package com.company.wqp07;
public class Dog extends Animal{
    @Override
```

```

    public void shout() {
        System.out.println("狗叫：汪汪!!!");
    }
    @Override
    public void eat() {
        System.out.println("狗吃骨头!!!");
    }
}
}
%-----%

package com.company.wqp07;
public class Test07 {
    public static void main(String[] args) {
        Cat cat1=new Cat();
        cat1.eat();
        cat1.shout();
        Dog dog1=new Dog();
        dog1.eat();
        dog1.shout();
    }
}

```

运行结果：

```

"C:\Program
猫吃鱼!!!
猫叫：喵喵!!!
狗吃骨头!!!
狗叫：汪汪!!!

```

图 1.7 运行结果示意图

8、编写程序，重写 toString()方法。

程序：

```

package com.company.wqp08;

/**
 * toString()方法没有重写之前；
 * return getClass().getName() +'@'+ Integer.toHexString(hashCode())
 * getClass() 获取当前运行对象的字节码对象。
 * getName() 获取包名和类名
 * @ 固定的连接符
 * hashCode() 获取模拟出来的内存地址值
 * Integer.toHexString(int num) 将十进制的整数转换成十六进制
 */
public class Student {
    String name;
    int age;
}

```

```
String num;

public Student() {
}

public Student(String name, int age, String num) {
    this.name = name;
    this.age = age;
    this.num = num;
}

@Override
public String toString() {
    return "name=" + name + ", age=" + age + ", num=" + num ;
}

}

%-----%

package com.company.wqp08;

public class Test08 {

    public static void main(String[] args) {
        Student stu1=new Student("李四",20,"20050090");
        System.out.println(stu1);
        System.out.println(stu1.toString());
    }

}
```

```
"C:\Program Files\Java\jdk-11\t  
name=李四, age=20, num=20050090  
name=李四, age=20, num=20050090
```

9、编写程序，演示 super 关键字的使用。

```
package com.company.wqp09;

/**
 * super 关键字
 * 可以访问父类的成员
 * 格式：
 * 1. 使用父类的成员变量：super. 成员变量
 * 2. 使用父类的成员方法：super. 成员方法([参数 1, 参数 2...])
 * 3. 调用父类的构造方法：super([参数 1, 参数 2...])
 */

public class Animal {
    String name;
    String color;
    int age;
    public Animal() {
```

```

    }
    public Animal(String name, String color, int age) {
        this.name = name;
        this.color = color;
        this.age = age;
    }
    public void eat() {
        System.out.println("动物会吃---!");
    }
    public void shout() {
        System.out.println("动物会叫---! ");
    }
}
%-----%

package com.company.wqp09;
public class Dog extends Animal{
    public Dog() {
        //调用父类的构造方法: super([参数 1, 参数 2...])
        super("斑点狗", "花色", 2);
    }
    @Override
    public void eat() {
        //使用父类的成员方法: super. 成员方法([参数 1, 参数 2...])
        super.eat();
        System.out.println("狗吃: 肉----! ");
    }
    @Override
    public void shout() {
        //使用父类的成员方法: super. 成员方法([参数 1, 参数 2...])
        super.shout();
        System.out.println("狗叫: 汪汪----! ");
    }
    @Override
    public String toString() {
        return "name="+name +", color=" + color + ", age=" + age;
    }
    public void print() {
        //使用父类的成员变量: super. 成员变量
        System.out.println(super.name);
    }
}
%-----%

package com.company.wqp09;
public class Test {

```

```

    public static void main(String[] args) {
        Dog dog=new Dog();
        dog.shout();
        dog.eat();
        System.out.println(dog.toString());
        dog.print();
    }
}

```

运行结果:



```

"C:\Program Files\Java\jdk1.8.
动物会叫---!
狗叫: 汪汪----!
动物会吃---!
狗吃: 肉----!
name=斑点狗, color=花色, age=2
输出名字: 斑点狗

```

图 1.9 运行结果示意图

10、编写程序，演示 final 关键字的使用。

程序:

```

package com.company.wqp10;

public /*final*/ class Animal {
    String name;
    String color;
    int age;
    public Animal() {
    }
    public Animal(String name, String color, int age) {
        this.name = name;
        this.color = color;
        this.age = age;
    }
    public /*final*/ void eat() {
        System.out.println("动物会吃---!");
    }
}

%-----%

package com.company.wqp10;

public class Dog extends Animal {
    @Override
    public void eat() {
        System.out.println("狗吃: 肉----!");
    }
}

```

```

    }
}
%-----%
package com.company.wqp10;

public class Test {
    public static void main(String[] args) {
        final int A=10;//变量就变成了一个常量，只能赋值一次
        //A=20;
        System.out.println(A);
    }
}

```

结论:

```

/**
 * final 关键字作用:
 * final 修饰类: 类变成了一个最终的类，不能有任何的子类
 * final 修饰方法: 方法变成了一个最终的方法，不能重写
 * final 修饰变量: 变量就变成了一个常量，只能赋值一次
 */

```

11、编写程序，学习抽象类的使用。

程序:

```

package com.company.wqp11;

/**
 * 抽象类:
 * 1. 什么是抽象类
 * 当我们进行父类抽取的时候，有些方法具体每个子类的实现方式都不太一样。
 * 那么这个时候，就应该把这个方法定义成抽象方法。有抽象方法的类一定是抽象。
 * 2. 抽象类和抽象方法如何定义
 * abstract
 * 3. 抽象类的成员特点
 * 成员变量: 既可以变量，也可以有常量。
 * 成员方法: 即可有抽象方法，也可以有非抽象方法。
 * 构造方法: 可以有构造方法。
 * 4. 抽象类的注意事项
 * 抽象类不能实例化对象。
 */
public abstract class Animal {
    String name;
    String color;
    int age;
    public Animal() {
    }

    public Animal(String name, String color, int age) {
    }
}

```

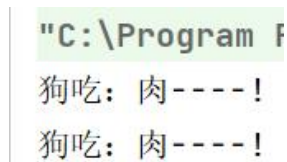
```

        this.name = name;
        this.color = color;
        this.age = age;
    }
    public abstract void eat();
}
%-----%
package com.company.wqp11;

public class Dog extends Animal {
    @Override
    public void eat() {
        System.out.println("狗吃：肉----!");
    }
}
%-----%
package com.company.wqp11;
public class Test11 {
    public static void main(String[] args) {
        //Animal an1=new Animal();抽象类不能实例化对象。
        Animal an2=new Dog();
        an2.eat();
        Dog an3=new Dog();
        an3.eat();
    }
}

```

运行结果：



```

"C:\Program F
狗吃：肉----!
狗吃：肉----!

```

图 1.10 运行结果示意图

12、编写程序，学习接口的使用。

程序：

```

package com.company.wqp12;
/**
 * 接口：
 * 1. 什么是接口
 * 接口是一种更加抽象的类。它完成的是一种特性的功能。
 * 接口的出现，打破了单继承的局限性。
 * 2. 接口如何定义及成员特点
 * public interface 接口名{
 *     //常量

```



```

* //抽象方法
* //默认方法
* //静态方法
* }
* 3. 接口的注意事项
* 接口不能有构造方法
* 接口不能直接创建对象使用的
* 4. 接口与类之间的关系
* 实现关系: implements 而且可以多实现
* 5. 接口和接口之间的关系
* 继承的关系: extends 接口可以多继承
*/
public interface Animal {
    public final int ID=10;
    public abstract void eat();
    public abstract void shout();
    public abstract void breath();
    public static int getId() {
        return ID;
    }
    public default void getType(String type) {
        System.out.println("当前动物是属于: "+type);
    }
}

%-----%
package com.company.wqp12;
public interface LandAnimal {
    public abstract void run();
}

%-----%
package com.company.wqp12;
public class Dog implements Animal, LandAnimal {
    @Override
    public void eat() {
        System.out.println("狗吃肉----!");
    }
    @Override
    public void shout() {
        System.out.println("狗汪汪的叫----!");
    }
    @Override
    public void breath() {
        System.out.println("狗在呼吸----!");
    }
}

```

```

    @Override
    public void run() {
        System.out.println("狗正在奔跑----!");
    }
}
%-----%

package com.company.wqp12;
public class Test {
    public static void main(String[] args) {
        Dog dog=new Dog();
        dog.shout();
        dog.breath();
        dog.eat();
        dog.run();
        dog.getType("犬科");
        Animal.getId();//静态方法不能被子类继承
    }
}

```

运行结果:

```

"C:\Program Files\
狗汪汪的叫----!
狗在呼吸----!
狗吃肉----!
狗正在奔跑----!
当前动物是属于: 犬科

```

图 1.11 运行结果示意图

13、编写程序，学习 Java 程序中的多态。

程序:

```

package com.company.wqp13;
/*
 * 多态
 * 1. 什么是多态
 * 指的是事物的多种表现形态。
 * 例如：猫是一只动物。猫是一只猫
 * 2. 多态的前提条件
 * 要有继承或者实现的关系
 * 要有方法重写
 * 要有父类引用指向子类的对象
 * 3. 多态的成员访问特点
 * 成员变量：编译看父类、运行看父类。
 * 成员方法：编译看父类、运行看子类。
 * 4. 多态的好处和弊端

```

- * 好处：提高代码扩展性和维护性
- * 弊端：无法使用子类特有的成员
- * 5. 多态的使用场景
- * 可以作为方法的参数和返回值进行使用。
- * 可以提高代码的扩展性。

*/

```
public abstract class Animal {
    String name;
    String color;
    int age=2;
    public Animal() {
    }

    public Animal(String name, String color, int age) {
        this.name = name;
        this.color = color;
        this.age = age;
    }
    public abstract void shout();
    public abstract void eat();
}

%-----%

package com.company.wqpl3;
public class Dog extends Animal{
    int age=10;
    @Override
    public void shout() {
        System.out.println("狗：汪汪汪的叫——！");
    }
    @Override
    public void eat() {
        System.out.println("狗吃肉——！");
    }
}

%-----%

package com.company.wqpl3;
public class Cat extends Animal{
    int age=20;
    @Override
    public void shout() {
        System.out.println("猫：喵喵喵的叫——！");
    }
    @Override
    public void eat() {
```

```

        System.out.println("猫吃鱼----!");
    }
}
%-----%
package com.company.wqpl3;
public class Test13 {
    public static void main(String[] args) {
        Animal an1=new Dog();//向上转型
        Animal an2=new Cat();
        method(an1);
        method(an2);
        System.out.println(an1.age);
        System.out.println(an2.age);
        Dog dog=(Dog)an1;//向下转型
        Cat cat=(Cat)an2;
        System.out.println(dog.age);
        System.out.println(cat.age);
    }
    public static void method(Animal d) {
        d.eat();
    }
}

```

运行结果:

```

"C:\Program
狗吃肉----!
猫吃鱼----!
2
2
10
20

```

图 1.12 运行结果示意图

14、编写程序，学习多态的转型，关键字 instanceof 的使用。

程序:

```

package com.company.wqpl4;
/**
 * 1. 多态的转型
 *   向上转型：父类引用指向子类对象
 *   向下转型：由父类引用转成一个对应的真实的子类对象
 *   格式：目标对象类型 对象名 = （目标对象类型）被转换的引用
 * 2. 注意事项：
 *   一定要确保转换的类型相同。否则会发类型转换异常：ClassCastException
 * 3. 关键字：
 *   instanceof 用于判断左边的引用是否是右边的对象类型

```

```

*/
public class Animal {
    int age=2;
    public void eat() {
        System.out.println("动物会吃饭---!!!");
    }
}

%-----%

package com.company.wqpl4;
public class Dog extends Animal{
    @Override
    public void eat() {
        System.out.println("狗吃肉---!!!");
    }
    public void lookHome() {
        System.out.println("狗会看家---!!!");
    }
}

%-----%

package com.company.wqpl4;
public class Cat extends Animal{
    int age=5;
    @Override
    public void eat() {
        System.out.println("猫吃鱼---!!!");
    }
    public void catchMouse() {
        System.out.println("猫会抓老鼠---!!!");
    }
}

%-----%

package com.company.wqpl4;
public class Test14 {
    public static void main(String[] args) {
        //向上转型 将一个子类对象赋值给了一个父类的引用对象引用
        Animal an1=new Dog();
        an1.eat();
        method(an1);
        Animal an2=new Cat();
        an2.eat();
        method(an2);
        // 由父类引用转成一个对应的真实的子类对象
        Cat cat=(Cat)an2;//向下转型
        cat.catchMouse();
    }
}

```

```

    }
    public static void method(Animal an) {
        if(an instanceof Dog) {
            Dog dog =(Dog) an;
            dog.lookHome();
        }else if(an instanceof Cat) {
            Cat cat =(Cat) an;
            cat.catchMouse();
        }
    }
}

```

运行结果:

```

"C:\Program File
狗吃肉---!!!
狗会看家---!!!
猫吃鱼---!!!
猫会抓老鼠---!!!
猫会抓老鼠---!!!

```

图 1.13 运行结果示意图

15、编写程序，学习如何定义成员内部类以及如何在外部类中访问内部类。

程序:

```

package com.company.wqp15;

/**
 * 创建内部类对象的格式:
 * 外部类.内部类 对象名 = new 外部类 ().new 内部类 ()
 * 成员内部类可以访问外部类的所有成员，无论外部类的成员是何种访问权限。
 */
public class Outer {
    //外部类的成员变量
    private int out = 10;
    //外部类的成员方法
    public void outerMethod() {
        System.out.println("我是外部类的成员方法");
    }
    //成员内部类
    class Inner{
        //内部类的成员变量
        int in=20;
        //内部类的成员方法
        public void innerMethod() {
            System.out.println("我是内部类的成员方法");
            System.out.println(out);
        }
    }
}

```

```

        outerMethod();
    }
}
}
%-----%
package com.company.wqp15;
/**
 * 外部类名 外部类对象 = new 外部类名();
 * 外部类名.内部类名 内部类对象 = 外部类对象.new 内部类名();
 */
public class Test15 {
    public static void main(String[] args) {
        // 如果想通过外部类访问内部类，则需要通过外部类创建内部类对象
        // Outer outer=new Outer();
        // Outer.Inner inner=outer.new Inner();
        Outer.Inner inner=new Outer().new Inner();
        inner.innerMethod();
        System.out.println(inner.in);
    }
}

```

运行结果:

```

"C:\Program Files'
我是内部类的成员方法
10
我是外部类的成员方法
20

```

图 1.14 运行结果示意图

16、编写程序，局部内部类的定义以及如何访问局部内部类。

程序:

```

package com.company.wqp16;
/**
 * 局部内部类可以访问外部类所有成员，而在外部类中无法直接访问局部内部类中的成员
 * 如何访问局部内部类？
 * 只需创建外部类对象。调用所属方法即可。
 */
public class Outer {
    //外部类的成员变量
    int outer = 10;
    //外部类的成员变量
    public void outerMethod() {
        System.out.println("我是外部类的成员方法");
    }
    public void function() {

```

```

//局部内部类
class Inner{
    //内部类的成员变量
    int inner = 20;
    //内部类的成员方法
    public void innerMethod() {
        System.out.println("我是内部类的成员方法");
        System.out.println(outer);
        outerMethod();
    }
}
//创建局部内部类对象
Inner i = new Inner();
System.out.println(i.inner);
i.innerMethod();
}
}
%-----%

package com.company.wqp16;
public class Test16{
    public static void main(String[] args) {
        Outer outer = new Outer();
        outer.outerMethod();
        outer.function();
    }
}
}
%-----%

```

运行结果:

```

"C:\Program Files\
我是外部类的成员方法
20
我是内部类的成员方法
10
我是外部类的成员方法

```

图 1.15 运行结果示意图

17、编写程序，学习静态内部类的定义和使用。

程序:

```

package com.company.wqp17;
/**
 * 静态成员内部类:
 * 静态内部类只能访问外部类的静态成员
 * 创建对象格式:

```



```

*      外部类名.内部类名 对象名 = new Outer.Inner();
*      外部类名.内部类名 对象名 = new Outer().new Inner();(普通成员内部类)
*/
public class Outer {
    //外部类的成员变量
    static int outer = 10;
    //外部类的成员方法
    public static void outerMethod() {
        System.out.println("我是外部类的成员方法");
    }
    //静态成员内部类
    static class Inner{
        //内部类的成员变量
        static int inner=20;
        //内部类的成员方法
        public static void innerMethod() {
            System.out.println("内部类的成员变量"+inner);
            System.out.println("外部类的成员变量"+outer);
            outerMethod();
        }
    }
}
}
%-----%
package com.company.wqp17;

public class Test17 {
    public static void main(String[] args) {
        Outer.Inner oi=new Outer.Inner();
        oi.innerMethod();
        Outer.Inner.innerMethod();
    }
}
}
%-----%

```

运行结果:

```

"C:\Program Files\
内部类的成员变量20
外部类的成员变量10
我是外部类的成员方法
内部类的成员变量20
外部类的成员变量10
我是外部类的成员方法

```

图 1.16 运行结果示意图

18、编写程序，学习匿名内部类的定义和使用。

程序：

```
package com.company.wqp18;

public abstract class Animal {
    public abstract void eat(); //传统的方式必须是子类重写
}

%-----%

package com.company.wqp18;

public class Dog extends Animal{
    @Override
    public void eat() {
        System.out.println("狗吃肉---!");
    }
}

%-----%

package com.company.wqp18;

/**
 * 匿名内部类的前提：必须是类或者接口
 * 格式：
 *     new 类名/接口名() { //子类或接口的实现类
 *         重写抽象方法
 *     }
 * 用途：作为方法的参数传递（前提是类或接口只有一个方法的时候），
 *     多个方法时，建议建立一个类实现接口或继承父类的形式。
 * 传统实现方式：
 *     1. 编写实现类
 *     2. 重写抽象方法
 *     3. 创建实现类对象
 *     4. 将实现类对象作为方法的参数传递
 */

public class Test18 {
    public static void main(String[] args) {
        /*传统的方式必须是子类重写，创建子类对象调用方法
        Dog d = new Dog();
        d.eat();*/
        useAnimal(new Animal() {
            @Override
            public void eat() {
                System.out.println("狗吃骨头----!");
            }
        });
        //调用方式一
        //整体就等效于：是 Animal 父类的子类对象
        new Animal() {
```

```

        @Override
        public void eat() {
            System.out.println("狗吃肉---");
        }
    }.eat();
    //调用方式二
    //通过匿名内部类访问局部变量。在 JDK8 版本之前，必须加 final 关键字
    String name = "哈士奇";
    //name = "金毛";//不能赋值
    Animal a = new Animal() {
        @Override
        public void eat() {
            System.out.println(name+"狗吃肉---");
        }
    };
    a.eat();
}

public static void useAnimal(Animal a){
    a.eat();
}
}

```

运行结果:

```

D:\Develop\Java
狗吃骨头----!
狗吃肉---
哈士奇狗吃肉---

```

图 1.17 运行结果示意图

19、控制台程序的输入输出处理。

编写 Java 应用程序，完成从键盘输入两个运算数据，计算两数之和并输出结果的功能。

程序 1:

```

package com.company.wqp19;
/**
 * 根据运行界面是以图形化要素为主，还是以文本字符为主，可分为两种主要类型：
 * GUI 应用程序(GUI Application)
 * 控制台应用程序(Console Application)
 */
import java.io.*; // 以便在程序中调用相关的类和方法
public class MyDemo01 {
    // 输入输出异常，由虚拟机自行处理
    public static void main(String args[]) throws IOException {
        // 输入数据
        // 控制台输入方案
    }
}

```

```

        System.out.println("输入一个整数: ");
        byte t[] = new byte[10];
        System.in.read(t); // 用于获取用户从键盘输入的数据。
        String s1 = new String(t);
        int a = Integer.parseInt(s1.trim()); // 去除空格
        System.out.println("输入一个小数: ");
        System.in.read(t);
        String s2 = new String(t);
        double b = Double.parseDouble(s2.trim());
        //处理数据
        double c = a + b;
        // 输出结果
        // 控制台输出方案
        System.out.println("计算的结果为: " + c);
    }
}
%-----%

```

程序 2:

```

package com.company.wqp19;
import java.util.Scanner;
/**
 * Scanner scan = new Scanner(System.in);
 * 是 Java 中用于从标准输入（通常是键盘）读取用户输入的代码。
 * 它的主要功能如下：
 * 创建 Scanner 对象：
 *   new Scanner(System.in)创建了一个 Scanner 类的实例，用于处理输入流。
 * 指定输入源：
 *   System.in 表示标准输入流，通常对应键盘输入，这意味着 Scanner 将从键盘读取用户输入的数据。
 * 提供多种输入方法：
 *   通过创建的 scan 对象，可以使用 Scanner 类提供的一系列方法读取不同类型的输入。
 * 例如：
 *       nextInt(): 读取整数
 *       nextDouble(): 读取双精度浮点数
 *       nextLine(): 读取一行文本
 *       next(): 读取一个单词（以空格为分隔符）
 *       使用完毕后，建议调用 scan.close()关闭 Scanner，以释放相关资源。
 */
public class MyDemo02 {
    /*两数求和*/
    public static void main(String args[]) {
        // 输入数据
        // 控制台输入方案
        Scanner scan = new Scanner(System.in);
    }
}

```

```

        System.out.println("输入一个整数: ");
        int a = scan.nextInt();
        System.out.println("输入一个小数: ");
        double b = scan.nextDouble();
        // 处理数据
        double c = a + b;
        // 输出结果
        // 控制台输出方案
        System.out.println("计算的结果为: " + c);
        // 使用完毕后, 建议调用 scan.close() 关闭 Scanner, 以释放相关资源。
    }
}

```

运行结果:



```

D:\Develop\Java\j
输入一个整数:
12
输入一个小数:
0.12
计算的结果为: 12.12

```

图 1.18 运行结果示意图

20、图形化程序的输入输出处理

编写 Java 程序, 利用图形化界面, 完成从键盘输入两个运算数据, 计算两数之和并输出结果的功能。

程序:

```

package com.company.wqp20;
import javax.swing.JOptionPane;
/**
 * 1. javax.swing 包中包含很多创建 Java 图形用户界面应用程序所必需的类。
 * 将 JOptionPane 类引入当前程序, 方便在程序中调用相关方法, 实现相关输入输出功能。
 * 2. 调用类 JOptionPane 的 showInputDialog 方法显示 “输入” 对话框。
 * showInputDialog 方法的参数为提示信息, 用以提示用户输入相关内容。
 * 在文本框中输入相关字符信息后, 单击 “确定” 按钮或按 Enter 键可以把文本框中的字符信息返回给 Java 程序。
 * 3. 调用类 JOptionPane 的 showMessageDialog 方法, 打开 “消息” 对话框显示结果信息。
 * 这个方法包含两个参数, 参数之间用逗号分隔。
 * 第一个参数表示对话框的父窗口对象, 当使用关键字 null 时, 表示对话框的父窗口不存在, 对话框将直接显示在计算机的显示器屏幕上;
 * 第二个参数为对话框中要显示的信息, 类型为字符串。
 */
public class MyDemo03 {
    public static void main(String args[])

```

```

{
    //输入数据
    //图形化输入方案
    String s1 = JOptionPane.showInputDialog("输入一个整数:");
    int a = Integer.parseInt(s1);
    String s2 = JOptionPane.showInputDialog("输入一个小数:");
    double b = Double.parseDouble(s2);
    //处理数据
    double c = a + b;
    //输出结果
    //图形化输出方案
    JOptionPane.showMessageDialog(null, "结果为:" + a + "+" + b + "=" + c);
}
}

```

运行结果:



图 1.19 运行结果示意图

21、计算圆的周长和面积

1) 编写 Java 控制台应用程序，完成从键盘输入圆的半径，计算圆的周长和面积并输出结果的功能。

程序:

```

package com.company.wqp21;
import java.io.*;
public class Circle {
    /*键盘输入半径，求圆的周长和面积*/
    public static void main(String[] args) throws IOException{
        // 定义常量 PI
        final double PI = 3.14;
        // 定义变量
        byte buf[] = new byte[50];
    }
}

```

```

double r,girth,area;
// 输入圆的半径
System.out.println("请输入一个圆的半径:");
System.in.read(buf);
String str = new String(buf);
r = Double.parseDouble(str.trim());
// 计算周长和面积
girth = 2 * PI * r;
area = PI * r * r;
// 输出结果
System.out.println("圆的周长为: " + girth);
System.out.println("圆的面积为: " + area);
}
}

```

运行结果:



```

D:\Develop\Java\j
请输入一个圆的半径:
3.5
圆的周长为: 21.98
圆的面积为: 38.465

```

图 1.20 运行结果示意图

%-----%

2) 编写 Java GUI 应用程序, 完成从键盘输入矩形的长和宽, 求矩形的周长和面积并输出结果的功能。

程序:

```

package com.company.wqp21;
import javax.swing.JOptionPane;
public class Circle {
    /*键盘输入长和宽, 计算矩形的周长和面积*/
    public static void main(String args[]){
        // 定义变量
        String s;
        double length,width,girth,area;
        // 输入矩形的长和宽
        s = JOptionPane.showInputDialog("输入矩形的长:");
        length = Double.parseDouble(s);
        s = JOptionPane.showInputDialog("输入矩形的宽:");
        width = Double.parseDouble(s);
        // 计算周长和面积
        girth = (length + width) * 2;
        area = length * width;
    }
}

```

```

// 输出计算结果
JOptionPane.showMessageDialog(null, "周长为: " + girth + "\n" + "面积为: "
+ area);
}
}

```

运行结果:

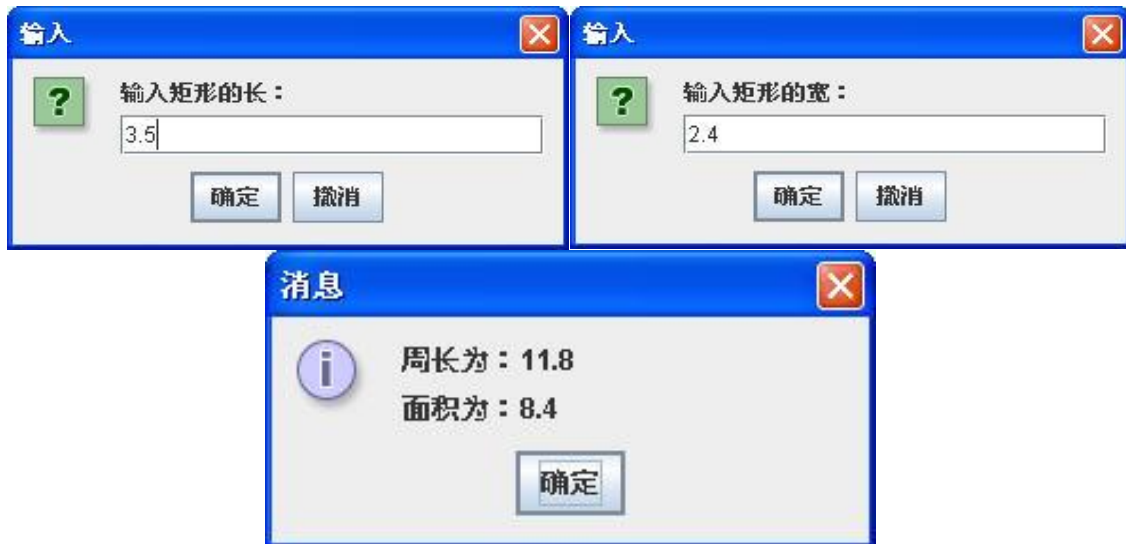


图 1.21 运行结果示意图

22、超市计价器的实现。

小明在超市购买了一瓶饮料和一个面包，请编写一个图形化的超市计价器程序，帮助他计算一下商品的总价格。

程序:

```

package com.company.wqp19;
import javax.swing.JOptionPane;
public class Calculator{
    public static void main(String[] args){
        //定义所需变量
        String temp;
        double oneprice, twoprice, totalprice;
        //输入两种商品的价格
        temp = JOptionPane.showInputDialog("请输入饮料的价格");
        oneprice = Double.parseDouble(temp);
        temp = JOptionPane.showInputDialog("请输入面包的价格: ");
        twoprice = Double.parseDouble(temp);
        //计算两种商品的总价格
        totalprice = oneprice + twoprice;
        //输出计算的总价格
        JOptionPane.showMessageDialog(null, "饮料价格: " + oneprice + "元\n" + "面包价格: " + twoprice + "元\n" + "商品总价格: " + totalprice + "元");
    }
}

```


运行结果：

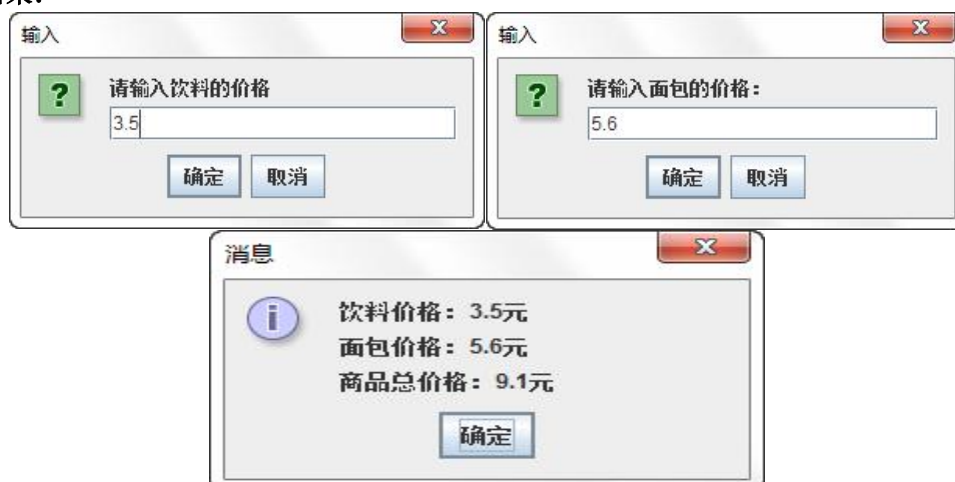


图 1.22 运行结果示意图

三、上机体会

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 程序的异常处理

实验时间 2025 年 10 月 10、17 日第 3、4 节

年级 2024 专业班级 计算机科学与技术1班

姓名 网球拍 学 号 20050090

评分：

教师签名： 王庆平

上机二 程序的异常处理

一、上机目的

- 1、了解什么是运行时异常和编译时异常，能够说出运行时异常和编译时异常的特点。
- 2、了解异常的产生及处理，能够说出处理异常的 5 个关键字。
- 3、掌握能够使用 try...catch 语句和 finally 语句处理异常。
- 4、掌握 throws 关键字的使用，能够使用 throws 关键字抛出异常。
- 5、掌握 throw 关键字的使用，能够使用 throw 关键字抛出异常。
- 6、掌握如何自定义异常，能够编写自定义异常类。

二、上机任务

- 1、编写程序，计算以 0 为除数的表达式，运行程序并观察程序的运行结果。

程序：

```
package com.company.computer01;

/**
 * 异常指程序运行过程中出现的非正常现象：
 * 例如文件找不到、用户输入错误、除数为零、数组下标越界等。
 * 异常是一个事件，它发生在程序运行期间，干扰了正常的指令流程。
 * Java 通过 API 中 Throwable 类的众多子类描述各种不同的异常。
 * Java 异常都是对象，是 Throwable 子类的实例，描述了出现在一段编码中的错误条件。
 * 当条件生成时，错误将引发异常。
 * Throwable：有两个重要的子类：
 *   Error（错误）：是指程序无法处理的错误，表示运行应用程序中较严重问题。
 *   Exception（异常）：是程序本身可以处理的异常，常分两大类：
 *   运行时异常和非运行时异常（编译异常）。程序中应当尽可能去处理这些异常。
 */
public class Example01 {
    public static void main(String[] args) {
        int result = divide(4, 0);    // 调用 divide() 方法，第 2 个参数为 0
        System.out.println(result);
    }
    //下面的方法实现了两个整数相除
    public static int divide(int x, int y) {
        int result = x / y;    // 定义一个变量 result 记录两个数相除的结果
        return result;        // 将结果返回
    }
}
```

运行结果：

```
D:\Develop\Java\jdk-11\bin\java.exe "-javaagent:D:\Develop\IntelliJ IDEA Community
Exception in thread "main" java.lang.ArithmeticException Create breakpoint : / by zero
at com.company.computer01.Example01.divide(Example01.java:20)
at com.company.computer01.Example01.main(Example01.java:15)
```

图 2.1 运行结果示意图

2、编写程序，使用 try...catch 语句对计算以 0 为除数的表达式出现的异常进行捕获。

程序：

```
package com.company.computer02;

/**
 * 异常处理，就是指程序在出现问题时依然可以正确的执行完。
 * 在 Java 应用程序中，异常处理机制分为：
 * 1) 抛出异常
 * 当一个方法出现错误引发异常时，方法创建异常对象并交付运行时系统；
 * 异常对象中包含了异常类型和异常出现时的程序状态等异常信息。
 * 运行时系统负责寻找处置异常的代码并执行。
 * 2) 捕捉异常。
 * 在方法抛出异常之后，运行时系统将转为寻找合适的异常处理器（exception handler）。
 * 潜在的异常处理器是异常发生时依次存留在调用栈中的方法的集合。
 * 通过 try-catch 语句捕获异常，这两个语句必须同时使用。其一般语法形式为：
 *
 * try {
 *     // 可能会发生异常的程序代码
 * } catch (异常类型 异常的变量名 1){
 *     // 捕获并处置 try 抛出的异常类型 1
 * } catch (异常类型 异常的变量名 2){
 *     // 捕获并处置 try 抛出的异常类型 2
 * }
 */
public class Example02 {
    public static void main(String[] args) {
        try{ // try 监控区域
            int result = divide(4, 0); //调用 divide()方法，第 2 个参数为 0
            System.out.println(result);
        }catch (ArithmeticException e){
            System.out.println(e.toString());
        }
        System.out.println("程序继续会执行下去-----");
    }
    //下面的方法实现了两个整数相除
    public static int divide(int x, int y) {
        int result = x / y; // 定义一个变量 result 记录两个数相除的结果
        return result; // 将结果返回
    }
}
```

运行结果：

```
D:\Develop\Java\jdk-11\bin\java.exe "-jav
java.lang.ArithmeticException: / by zero
程序继续会执行下去-----"
```

图 2.2 运行结果示意图

3、编写程序，演示 try...catch...finally 语句块的使用。

程序：

```
package com.company.computer03;

/**
 * try-catch-finally 语句：
 * try 块：用于捕获异常。
 * 其后可接零个或多个 catch 块，如果没有 catch 块，则必须跟一个 finally 块。
 * catch 块：用于处理 try 捕获到的异常。
 * finally 块：无论是否捕获或处理异常，finally 块里的语句都会被执行。
 * 当在 try 块或 catch 块中遇到 return 语句时，finally 语句块将在方法返回之前被执行。
 * try-catch-finally 语句的一般语法形式为：
 * try{
 *     // 可能会发生异常的程序代码
 * }catch(Type1 id1){
 *     // 捕获并处理 try 抛出的异常类型 Type1
 * }catch(Type2 id2){
 *     // 捕获并处理 try 抛出的异常类型 Type2
 * }finally{
 *     // 无论是否发生异常，都将执行的语句块
 * }
 * 在以下 4 种特殊情况下，finally 块不会被执行：
 * 1) 在 finally 语句块中发生了异常；
 * 2) 在前面的代码中用了 System.exit() 退出程序。
 * 3) 程序所在的线程死亡。
 * 4) 关闭 CPU。
 * try、catch、finally 语句块的执行顺序：
 * 1) 当 try 没有捕获到异常时：try 语句块中的语句逐一被执行，程序将跳过 catch 语句块，执行 finally 语句块和其后的语句；
 * 2) 当 try 语句块里的某条语句出现异常时，而没有处理此异常的 catch 语句块时，此异常将会抛给 JVM 处理，finally 语句块里的语句还是会被执行，但 finally 语句块后的语句不会被执行；
 * 3) 当 try 捕获到异常，catch 语句块里有处理此异常的情况：在 try 语句块中是按照顺序来执行的，当执行到某一条语句出现异常时，程序将跳到 catch 语句块，并与 catch 语句块逐一匹配，找到与之对应的处理程序，其他的 catch 语句块将不会被执行，而 try 语句块中，出现异常之后的语句也不会被执行，catch 语句块执行完后，执行 finally 语句块里的语句，最后执行 finally 语句块后的语句。
 */
public class Example03 {
    public static void main(String[] args) {
        try{
            int result = divide(4, 0); //调用 divide() 方法，第 2 个参数为 0
            System.out.println(result);
        }catch (ArithmeticException e){
            System.out.println(e.toString());
        }
    }
}
```

```

        return;
    }finally {
        System.out.println("程序继续会执行下去-----");
    }
}
//下面的方法实现了两个整数相除
public static int divide(int x, int y) {
    int result = x / y;    // 定义一个变量 result 记录两个数相除的结果
    return result;        // 将结果返回
}
}

```

运行结果:

```

D:\Develop\Java\jdk-11\bin\java.exe "-jav
java.lang.ArithmeticException: / by zero
程序继续会执行下去-----

```

图 2.3 运行结果示意图

4、编写程序，演示 throws 关键字的使用。

程序:

```

package com.company.computer04;
/**
 * 修饰符 返回值类型 方法名(参数 1, 参数 2, ...)throws 异常类 1, 异常类 2... {
 *      方法体
 * }
 * throws 关键字声明该方法有可能发生的异常，这样调用者在调用方法时，
 * 就明确地知道该方法有异常，并且必须在程序中对异常进行处理，否则编译无法通过。
 */
public class Example04 {
    public static void main(String[] args) {
        // 下面的代码定义了一个 try...catch 语句用于捕获异常
        try {
            int result = divide(4, 2);    // 调用 divide() 方法
            System.out.println("4÷2="+result);
        } catch (Exception e) {          // 对捕获到的异常进行处理
            e.printStackTrace();          // 打印捕获的异常信息
        }
    }
    // 实现了两个整数相除，并使用 throws 关键字声明抛出异常
    public static int divide(int x, int y) throws Exception {
        int result = x / y;    // 定义变量 result 记录两个数相除的结果
        return result;        // 将结果返回
    }
}

```

运行结果：

```
D:\Devel
4÷2=2
```

图 2.4 运行结果示意图

5、编写程序，演示 throw 关键字的使用。

程序：

```
package com.company.computer05;
/**
 * throw 总是出现在函数体中，用来抛出一个 Throwable 类型的异常。
 * 程序会在 throw 语句后立即终止，它后面的语句执行不到，
 * 然后在包含它的所有 try 块中（可能在上层调用函数中）从里向外寻找含有与其匹配的
 * catch 子句的 try 块。
 */
public class Example05 {
    public static void main(String[] args) {
        // 下面的代码定义了一个 try...catch 语句用于捕获异常
        int age = -1;
        try {
            printAge(age);
        } catch (Exception e) { // 对捕获到的异常进行处理
            System.out.println("捕获的异常信息为：" + e.getMessage());
        }
    }
    public static void printAge(int age) throws Exception {
        if(age <= 0) {
            // 对业务逻辑进行判断，当输入年龄为负数时抛出异常
            throw new Exception("输入的年龄有误，必须是正整数！");
        } else {
            System.out.println("此人年龄为：" + age);
        }
    }
}
```

运行结果：

```
D:\Develop\Java\jdk-11\bin\java.exe "-j
捕获的异常信息为：输入的年龄有误，必须是正整数！
```

图 2.5 运行结果示意图

6、编写程序，自定义的异常，在 divide() 方法中判断被除数是否为负数，如果为负数，就使用 throw 关键字在方法中向调用者抛出自定义的 DivideByMinusException 异常对象。

程序：

```
package com.company.computer06;
/**
 * 自定义的异常类只需继承 Exception 类，在构造方法中使用 super() 语句调用 Exception
```

* 的构造方法即可。

*/

```
public class DivideByMinusException extends Exception {
    public DivideByMinusException() {
        super();
    }
    public DivideByMinusException(String message) {
        super(message);
    }
}
```

%-----%

```
package com.company.computer06;
```

```
public class Example06 {
    public static void main(String[] args) {
        try{
            System.out.println(divide(-4, 2));
        }catch (DivideByMinusException e){
            System.out.println(e.toString());
        }
        System.out.println("程序会继续执行下去-----!!!");
    }
    public static int divide(int x,int y)throws DivideByMinusException{
        if (x<0)
            throw new DivideByMinusException("被除数是负数的异常---!");
        else {
            int result = x/y;
            return result;
        }
    }
}
```

运行结果:

```
D:\Develop\Java\jdk-11\bin\java.exe "-javaagent:D:\Develop\Intelli
com.company.computer06.DivideByMinusException: 被除数是负数的异常---!
程序会继续执行下去-----!!!
```

图 2.6 运行结果示意图

三、上机体会

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 集合

实验时间 2025 年 10 月 24、31 日、11 月 7 日第 3、4 节

年级 2024 专业班级 计算机科学与技术 1 班

姓名 网球拍 学 号 20050090

评分：

教师签名： 王庆平

上机三 集合

一、上机目的

- 1、了解集合的概念,能够说出集合用于做什么。
- 2、熟悉 Collection 接口,能够说出 Collection 接口中的常用方法。
- 3、掌握 List 接口、Set 接口、Map 接口的使用。
- 4、掌握 List 接口中的 ArrayList、LinkedList、Iterator 接口和 foreach 循环的使用。
- 5、掌握 Set 接口中的 HashSet、LinkedHashSet 和 TreeSet 的使用。

二、上机任务

- 1、编写程序,演示 Collection 接口中常用方法的使用。

程序:

```
package com.company.com01;
import java.util.*;
public class CollectionTest {
    public static void main(String[] args) {
        // 泛型: 约束集合可以存储数据的类型
        Collection<String> c1 = new ArrayList<>();
        // boolean add(Object o)    添加元素
        c1.add("aa");c1.add("bb");System.out.println(c1);
        // boolean addAll(Collection c)    将指定集合中所有元素添加到当前集合
        Collection<String>c2 = new ArrayList<>();
        c2.add("cc");c2.add("dd");c1.addAll(c2);
        System.out.println(c1);
        // boolean contains(Object o)    判断集合是否包含指定元素
        System.out.println(c1.contains("aa"));
        System.out.println(c1.contains("aaa"));
        // boolean containsAll(Collection c)    判断集合中是否包含指定集合所有元素
        System.out.println(c1.containsAll(c2));
        // int size()    获取集合的长度
        System.out.println(c1.size());
    }
}
```

运行结果:

```
D:\Develop\Java\
[aa, bb]
[aa, bb, cc, dd]
true
false
true
4
```

图 3.1 运行结果示意图

2、编写程序，演示 list 接口中常用方法的使用。

程序：

```
package com.company.com02;
import java.util.ArrayList;
import java.util.List;
public class ListDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("bb");
        list.add("cc");
        //void add(int index, Object element) 将指定索引位置添加元素
        list.add(0, "aa");
        System.out.println(list);
        List<String> list2 = new ArrayList<>();
        list2.add("dd");
        list2.add("cc");
        //addAll(int index, Collection c) 将指定集合元素添加到当前集合指定索引位置
        list.addAll(0, list2);
        System.out.println(list);
        //Object get(int index) 获取指定索引处的元素
        System.out.println(list.get(0));
        //Object remove(int index) 删除指定索引处的元素，返回被删除的元素
        System.out.println(list.remove(0));
        System.out.println(list);
        //Object set(int index, Object obj) 修改指定索引处元素，返回被修改的元素
        System.out.println(list.set(0, "qq"));
        System.out.println(list);
    }
}
```

运行结果：

```
D:\Develop\Java\jdk-1
[aa, bb, cc]
[dd, cc, aa, bb, cc]
dd
dd
[cc, aa, bb, cc]
cc
[qq, aa, bb, cc]
```

图 3.2 运行结果示意图

3、编写程序，演示 ArrayList 集合的元素存取。

程序：

```
package com.company.com03;
import java.util.ArrayList;
```

```

/**
 * ArrayList 集合看作一个长度可变的数组。
 * ArrayList 特点：
 * 底层是数组结构实现的，数组有索引、查询速度比较快。
 * 在增加或删除指定位置的元素时，会创建新的数组，增删比较慢。
 * 使用场景：如果大量的数据经常做查询的操作，优先使用 ArrayList*/
public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        // 向集合中添加元素，有序，可重复
        list.add("张三");
        list.add("张三");
        list.add("李四");
        list.add("王五");
        list.add("赵六");
        System.out.println(list);
        // 获取集合中元素的个数
        System.out.println("集合的长度：" + list.size());
        // 取出并打印指定位置的元素
        System.out.println("第 2 个元素是：" + list.get(1));
        list.remove(3); // 删除索引为 3 的元素
        System.out.println("删除索引为 3 的元素：" + list);
        list.set(1, "李四 2"); // 替换索引为 1 的元素为李四 2
        System.out.println("替换索引为 1 的元素为李四 2：" + list);
    }
}

```

运行结果：

```

D:\Develop\Java\jdk-11\bin\java.exe "-javaa
[张三, 张三, 李四, 王五, 赵六]
集合的长度: 5
第2个元素是: 张三
删除索引为3的元素:[张三, 张三, 李四, 赵六]
替换索引为1的元素为李四2:[张三, 李四2, 李四, 赵六]

```

图 3.3 运行结果示意图

4、编写程序，演示 LinkedList 集合的元素增加、删除和获取操作。

程序：

```

package com.company.com04;
import java.util.LinkedList;
/**
 * LinkedList 集合内部维护了一个双向循环链表，链表中的每一个元素都使用引用的方式
 * 记录它的前一个元素和后一个元素，
 * 从而可以将所有的元素彼此连接起来。当插入一个新元素时，只需要修改元素之间的引

```

用关系即可，删除一个节点也是如此。

* 正因为这样的存储结构，所以 LinkedList 集合增删效率非常高。

*/

```
public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("张三");
        list.add("李四");
        list.add("王五");
        System.out.println(list);
        list.addFirst("赵六");    // 添加元素到开头
        System.out.println(list);
        list.addLast("周七");    // 添加元素到结尾
        System.out.println(list);
        String first = list.getFirst(); // 获取第一个元素并返回
        System.out.println(first);
        String last = list.getLast();    // 获取最后一个元素并返回
        System.out.println(last);
        System.out.println(list.removeFirst()); // 删除第一个元素并返回
        System.out.println(list.removeLast()); // 删除最后一个元素并返回
        System.out.println(list);
    }
}
```

运行结果:

```
D:\Develop\Java\jdk-11\bin
[张三, 李四, 王五]
[赵六, 张三, 李四, 王五]
[赵六, 张三, 李四, 王五, 周七]
赵六
周七
赵六
周七
[张三, 李四, 王五]
```

图 3.4 运行结果示意图

5、编写程序，演示遍历集合中的元素。

程序:

```
package com.company.com05;
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorDemo {
    public static void main(String[] args) {
        ArrayList<String> list =new ArrayList<>();
        list.add("张三");
```

```

list.add("李四");
list.add("王五");
// 第 1 种方法:
System.out.println(list);
// 第 2 种方法: for 循环
for(int i = 0;i<list.size();i++){
    // list 集合有索引, 可以用 for 来遍历
    String s = list.get(i);
    System.out.print(s+" ");
}
System.out.println();
//第 3 种方法: 增强 for 循环 (foreach 循环)
for(String str:list){
    System.out.print(str+" ");
    str="qq";
    //str 是第三方变量, 用来依次接收集合中的数据, 改变不会影响原集合中值
}
System.out.println();
//第 4 种方法: 迭代器是通用的遍历方式
Iterator<String> it = list.iterator(); // 1. 获取迭代器对象
while(it.hasNext()){ // 2. 判断迭代器中是否还有下一个元素
    Object str=it.next();// 3. 获取迭代器中下一个元素
    System.out.print(str+" ");
}
}
}

```

运行结果:

```

D:\Develop\Java\;
[张三, 李四, 王五]
张三 李四 王五
张三 李四 王五
张三 李四 王五

```

图 3.5 运行结果示意图

6、编写程序, 演示向 HashSet 集合中添加数据时去除重复数据。

程序 1:

```

package com.company.com06;
import java.util.HashSet;
import java.util.Iterator;
/**
 * HashSet 集合: 没有索引、不能存储重复元素、元素唯一存取无序
 * 元素唯一的原理:
 * 1、根据对象的哈希值(调用 hashCode() 计算)来计算存储位置
 * 2、判断当前位置时候有数据

```

```

*      2.1、没有：直接存储
*      2.2、有：调用 equals() 方法比较属性值
*          2.2.1、属性值相同：不存储
*          2.2.2、属性值不同：以链表结构存储
* */
public class HashSetDemo01 {
    public static void main(String[] args) {
        HashSet<String> hs = new HashSet<>();
        hs.add("张三");
        hs.add("张三");
        hs.add("李四");
        hs.add("王五");
        hs.add("赵六");
        /*for(int i = 0; i<hs.size;i++) {
            hs.get(i); 没有索引，报错，不能用 for 循环遍历
        }*/
        for (String s : hs) {
            System.out.print(s+" ");
        }
        System.out.println();
        Iterator<String> it = hs.iterator(); // 获取迭代器对象
        while(it.hasNext()){
            String str=it.next();
            System.out.print(str+" ");
        }
    }
}

```

运行结果 1:

```

D:\Develop\Java\j
李四 张三 王五 赵六
李四 张三 王五 赵六

```

图 3.6 运行结果示意图

%-----%

程序 2:

```

package com.company.com06;
/**
 * Set 集合去重原理:
 * 1. 调用对象的 hashCode 方法，得到哈希值
 *    如果哈希值不同，则直接存储
 * 2. 当哈希值相同时，才会调用 equals 方法
 *    比较结果为 false 则存储
 *    比较结果是 true 才会当成重复元素，不存
 * 如何把内容相同视为相同元素:

```

```

*    解决方案:
*    重写 hashCode 和 equals 方法
* */
import java.util.Objects;
public class Student /*extends Object*/{
    private String name;
    private int age;
    public Student() {
    }
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student student = (Student) o;
        return age == student.age && Objects.equals(name, student.name);
    }
    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }
    @Override
    public String toString() {
        return "名字: " + name + ", 年龄=" + age;
    }
}
%-----%

package com.company.com06;
import java.util.HashSet;
public class HashSetDemo02 {
    public static void main(String[] args) {
        //注意:如果是自定义类型的对象, 保存到 HashSet 集合,
        //必须要重写 hashCode() 和 equals() 方法。
        HashSet<Student> hs = new HashSet<>();
        Student s1 = new Student("张三", 23);
        Student s2 = new Student("张三", 23);
        Student s3 = new Student("李四", 24);
        Student s4 = new Student("王五", 25);
        hs.add(s4);
        hs.add(s1);
        hs.add(s2);
    }
}

```



```

        hs.add(s3);
        for (Student student : hs) {
            System.out.println(student);
        }
    }
}

```

运行结果 2:

```

D:\Develop\Java\jdk-8.0.601\bin
名字: 张三, 年龄=23
名字: 李四, 年龄=24
名字: 王五, 年龄=25

```

图 3.7 运行结果示意图

7、编写程序，学习 LinkedHashMap 类的用法。

程序:

```

package com.company.com07;

/**
 * HashSet 集合存储的元素是无序的
 * 想让元素的存取顺序一致，可使用 LinkedHashMap 类，是 HashSet 的子类
 * 与 LinkedList 一样，它也使用双向链表来维护内部元素的关系。
 */
import java.util.Iterator;
import java.util.LinkedHashSet;
public class LinkedHashMapDemo {
    public static void main(String[] args) {
        LinkedHashMap set = new LinkedHashMap();
        set.add("张三");           // 向该 Set 集合中添加字符串
        set.add("李四");
        set.add("王五");
        Iterator it = set.iterator(); // 获取 Iterator 对象
        while (it.hasNext()) {        // 通过 while 循环，判断集合中是否有元素
            Object obj = it.next();
            System.out.println(obj);
        }
    }
}

```

运行结果:

```

D:\De
张三
李四
王五

```

图 3.8 运行结果示意图

8、编写程序，演示 TreeSet 的排序规则，自然排序和自定义排序。

程序 1:

```
package com.company.com08;

/**
 * TreeSet 集合：底层是二叉树实现。可以对存入的元素进行排序。
 * 排序方式一：自然排序。要求集合中元素的类必须要实现 Comparable 接口，
 * 重写 compareTo 编写比较的条件。
 * Java 中大部分的类都实现了 Comparable 接口，并默认实现了接口中的 compareTo() 方法，
 * 如 Integer、Double 和 String 等。
 * 规则：从小到大
 */
import java.util.TreeSet;
public class TreeSetDemo01 {
    public static void main(String[] args) {
        TreeSet<Integer> ts1 = new TreeSet<>();
        ts1.add(5);
        ts1.add(3);
        ts1.add(2);
        ts1.add(1);
        ts1.add(4);
        System.out.println(ts1); //[1, 2, 3, 4, 5]
        TreeSet<String> ts2 = new TreeSet<>();
        ts2.add("Tom");
        ts2.add("Candy");
        ts2.add("Baby");
        ts2.add("David");
        for (Object o:ts2) {
            System.out.println(o);
        }
    }
}
```

运行结果 1:

```
D:\Develop\Java
[1, 2, 3, 4, 5]
Baby
Candy
David
Tom
```

图 3.9 运行结果示意图

程序 2:

```

package com.company.com08;

/**
 * TreeSet 集合：底层是二叉树实现。可以对存入的元素进行排序。
 * 排序方式一：自然排序。要求集合中元素的类必须要实现 Comparable 接口，
 * 重写 compareTo 编写比较的条件。
 * Java 中大部分的类都实现了 Comparable 接口，并默认实现了接口中的 compareTo() 方法，
 * 如 Integer、Double 和 String 等，规则：从小到大。
 * 对自定义对象进行排序？
 * 自定义类：要求存储的元素类必须实现 Comparable 接口，并重写 compareTo() 方法。
 * 可不可以从大到小？
 */
public class Student implements Comparable<Student>{
    private String name;
    private int age;
    public Student() {
    }
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
    @Override
    public int compareTo(Student o) { // 排序方式一：自然排序
        //排序规则：按照年龄升序
        int result = this.age-o.getAge(); //o.getAge()-this.age 降序
        /* 编写排序规则返回值特点：
        1. 返回的是负数（小）、向左存
        2. 返回的是 0、不存储
        3. 返回的是正数（大）、向右存

```

```

    */
    //次要条件：如果年龄相同、按照姓名排序
    if (result==0) {
        result = this.name.compareTo(o.getName());
    }
    return result;
}
}
}
%-----%

package com.company.com08;
import java.util.TreeSet;
/**
 * TreeSet 集合：底层是二叉树实现。可以进行元素的排序
 * 自定义类：要求存储的元素类必须实现 Comparable 接口，并重写 compareTo() 方法。
 */
public class TreeSetDemo02 {
    public static void main(String[] args) {
        TreeSet<Student> ts = new TreeSet<>();
        Student s1 = new Student("aa",25);
        Student s2 = new Student("aa",25);
        Student s3 = new Student("bb",23);
        Student s4 = new Student("ba",23);
        Student s5 = new Student("cc",24);
        ts.add(s1);
        ts.add(s2);
        ts.add(s3);
        ts.add(s4);
        ts.add(s5);
        //System.out.println(ts);
        for (Student student : ts) {
            System.out.println(student);
        }
    }
}
}

```

运行结果 2:

```

D:\Develop\Java\
name=ba, age=23
name=bb, age=23
name=cc, age=24
name=aa, age=25

```

图 3.10 运行结果示意图

程序 3:

```
package com.company.com08;
public class Teacher {
    private String name;
    private int age;
    public Teacher() {
    }
    public Teacher(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    @Override
    public String toString() {
        return "name=' " + name + "', age=" + age;
    }
}
%-----%

package com.company.com08;
import java.util.Comparator;
import java.util.TreeSet;
/**
 * 排序方式二（推荐）：比较器排序。需要在 TreeSet 集合的构造方法中, 传递比较器接口的实现类对象。编写排序条件！
 *
 * TreeSet 排序：
 * 自然排序：
 * 自定义类 实现 Comparable 接口重写 compareTo 方法并指定排序规则
 * 定制排序：（推荐）
 * 通过 TreeSet 的有参构造，传递 Comparator 对象并指定排序规则
 * 注意：定制排序优先级大于自然排序
 */
public class TreeSetDemo03 {
```

```

public static void main(String[] args) {
    TreeSet<Teacher> ts = new TreeSet<Teacher>(new Comparator<Teacher>() {
        @Override
        public int compare(Teacher o1, Teacher o2) {
            //按照年龄降序排序
            int result = o2.getAge() - o1.getAge();
            return result;
        }
    });
    Teacher t1 = new Teacher("张三", 23);
    Teacher t2 = new Teacher("王五", 25);
    Teacher t3 = new Teacher("李四", 26);
    Teacher t4 = new Teacher("赵四", 24);
    ts.add(t1);
    ts.add(t2);
    ts.add(t3);
    ts.add(t4);
    //System.out.println(ts);
    for (Teacher teacher: ts) {
        System.out.println(teacher);
    }
}
}

```

运行结果 3:

```

D:\Develop\Java\jdk
name='李四, age=26
name='王五, age=25
name='赵四, age=24
name='张三, age=23

```

图 3.10 运行结果示意图

9、编写程序，演示 Map 接口常用方法。

程序：

```

package com.company.com09;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
/**
    Map 接口方法：
    void put(Object key, Object value)    添加一对数据
    Object get(Object key)                根据键获取值
    void clear()                          清空集合元素
    Object remove(Object key)             根据键删除一对数据，返回被删除的值

```

```

    int size()                获取集合中的长度
    boolean containsKey(Object key)  判断集合中是否包含指定的键
    boolean containsValue(Object value)  判断集合中是否包含指定的值
    Collection<V> values()      获取集合中所有的值，保存到单列集合
    String replace (Object key, Object value) 将集合中 key 所映射的值修改为 value
*/
public class MapDemo {
    public static void main(String[] args) {
        //创建 HashMap 集合对象
        //Map<String,String> map = new HashMap<>();
        HashMap<String,String> map = new HashMap<>();
        //void put(Object key, Object value) 添加一对数据（键值映射）
        //Map 集合中的键是唯一的、值可以重复
        map.put("s01","张三");
        map.put("s02","王五");
        map.put("s03","赵六");
        //使用新值将老值替换。并返回老值
        String oldValue =map.put("s01","李四");
        System.out.println(oldValue);
        System.out.println(map);
        //有时内容不同，哈希值相同
        //System.out.println("重地".hashCode());
        //System.out.println("通话".hashCode());
        //method01(map);
    }
    private static void method01(Map<String, String> map) {
        //int size() 获取集合中的长度
        System.out.println(map.size());
        //Object get(Object key) 根据键获取值
        String value = map.get("s02");
        System.out.println(value);
        //boolean containsKey(Object key) 判断集合中是否包含指定的键
        System.out.println(map.containsKey("s02"));
        System.out.println(map.containsKey("s05"));
        //boolean containsValue(Object value) 判断集合中是否包含指定的值
        System.out.println(map.containsValue("王五"));
        System.out.println(map.containsValue("周七"));
        //V remove(Object key) 根据键删除一对数据，返回被删除的值
        String value2 = map.remove("s03");
        System.out.println(value2);
        System.out.println(map);
        //Collection<V> values() 获取集合中所有的值，保存到单列集合
        Collection<String> values = map.values();
        System.out.println(values);
    }
}

```

```

        //String replace (Object key,Object value)
        //将集合中 key 所映射的值修改为 value
        map.replace("s02","田老八");
        System.out.println(map);
        //void clear()                清空集合元素
        map.clear();
        System.out.println(map);
    }
}

```

运行结果:

```

D:\Develop\Java\jdk-11\bin\ja
张三
{s02=王五, s01=李四, s03=赵六}

```

图 3.11 运行结果示意图

10、编写程序，演示 Map 集合遍历方式。

程序:

```

package com.company.com10;
import java.util.*;
/**
 * Map 遍历:
 * map 集合是不能直接使用增强 for 和普通 for，迭代器.
 * Map 集合遍历方式:
 * 方法一: Set keySet()    获取所有的键、保存到 Set 集合
 * 1、map 双列集合不能直接遍历
 * 2、调用 keySet() 获取所有的键，保存到 Set 集合中，
 * 遍历 Set 集合可获得所有的键，配合 get() 方法就可获得键对应的值。
 * 方法二: Set<Map.Entry<K,V>>entrySet()  获取所有键值对对象、保存到 Set 集合
 * entrySet() 方法会将每一对数据，封装成一个一个 Entry 对象，
 * 然后保存在 Set 集合中，然后遍历 Set 集合，得到 Entry 对象，
 * 通过 getKey() 和 getValue() 方法获得键和值。
 * 第三种: forEach (BiConsumer) JDK8 版本新增的遍历的方法。
 *
 * 第四种: values () ; 获取 map 集合中所有的值，返回到一个 Collection 集合中。
 *      forEach () ;
 */
public class LinkedHashMapDemo {
    public static void main(String[] args) {
        //创建 HashMap 集合对象
        //Map<String,String> map = new HashMap<>();
        HashMap<String,String> map = new HashMap<>();
        // void put(Object key, Object value) 添加一对数据 (键值映射)
    }
}

```



```

map.put("hm001", "张三");//Map 集合中的键是唯一的、值可以重复
map.put("hm002", "李四");
map.put("hm003", "王五");
map.put("hm004", "赵六");
// 第一种方式: keySet() 方法
// Set keySet() 获取所有的键、保存到 Set 集合
Set<String> keys = map.keySet(); //map 双列集合不能直接遍历, 两种方式
for (String key : keys) {
    String value = map.get(key);
    System.out.println(key+", "+value);
}
System.out.println("-----");
/*
// 第二种方式: entrySet() 方法
// Set<Map.Entry<K, V>>entrySet() 获取所有键值对对象、保存到 Set 集合
Set<Map.Entry<String, String>> entries = map.entrySet();
for (Map.Entry<String, String> entry : entries) {
    String key = entry.getKey();
    String value = entry.getValue();
    System.out.println(key+", "+value);
}
System.out.println("-----");
Iterator<Map.Entry<String, String>> it = entries.iterator();
while (it.hasNext()) {
    Map.Entry<String, String> entry = it.next();
    String key = entry.getKey();
    String value = entry.getValue();
    System.out.println(key+", "+value);
}
System.out.println("-----");
//第三种: forEach (BiConsumer) JDK8 版本新增的遍历的方法。
map.forEach((String key, String value)->{
    System.out.println(key+"="+value);
});
System.out.println("-----");*/
//第四种: values (); 获取 map 集合中所有的值, 返回到一个 Collection 集合中。
//forEach ();
Collection<String> values = map.values();
values.forEach((String value)->{
    System.out.println(value);
});
}
}

```

运行结果:

```
D:\Develop'  
hm004,赵六  
hm003,王五  
hm002,李四  
hm001,张三  
-----  
赵六  
王五  
李四  
张三
```

图 3.12 运行结果示意图

11、编写程序，学习 LinkedHashMap 集合的用法。

程序：

```
package com.company.com11;  
import java.util.LinkedHashMap;  
import java.util.Set;  
public class LinkedHashMapTest {  
    public static void main(String[] args) {  
        //HashMap<String,String> hm = new HashMap<>(); //存储无序  
        //注意：如果想要保证元素的存取顺序，可以使用 LinkedHashMap 集合。  
        LinkedHashMap<String,String> hm = new java.util.LinkedHashMap<>();  
        hm.put("s01","张三");  
        hm.put("s02","李四");  
        hm.put("s03","王五");  
        hm.put("s04","赵六");  
        Set<String> keys = hm.keySet();  
        for (String key : keys) {  
            String value = hm.get(key);  
            System.out.println(key+", "+value);  
        }  
    }  
}
```

运行结果：

```
D:\Develo  
s01,张三  
s02,李四  
s03,王五  
s04,赵六
```

图 3.13 运行结果示意图

12、编写程序，实现 TreeMap 集合按键值排序，键是自定义类，值是 String 类。

程序：

```
package com.company.com12;

/**
 * 自定义类：要求存储的元素类必须实现 Comparable 接口，并重写 compareTo() 方法。
 */
public class Student implements Comparable<Student>{
    private String name;
    private int age;
    public Student() {
    }
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age="+age;
    }
    @Override
    public int compareTo(Student o) {
        //排序规则：按照年龄升序
        int result = o.getAge()-this.age; //this.age-o.getAge();升序
        //次要条件：如果年龄相同、按照姓名排序
        if (result==0){
            result = this.name.compareTo(o.getName());
        }
        return result;
    }
}

%-----%
```

```

package com.company.com12;
public class Teacher {
    private String name;
    private int age;
    public Teacher() {
    }
    public Teacher(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
}
%-----%

package com.company.com12;
import java.util.Comparator;
import java.util.Iterator;
import java.util.Set;
import java.util.TreeMap;
public class TreeMapTest {
    public static void main(String[] args) {
        //方法 1: 自然排序升序, String 类已经重写 compareTo 方法了。
        TreeMap<String, String> tm = new TreeMap<>();
        tm.put("hm002", "张三");
        tm.put("hm003", "李四");
        tm.put("hm001", "王五");
        tm.put("hm004", "赵六");
        System.out.println(tm);
        //方法 1: 重写 compareTo 方法降序排序。
        TreeMap<Student, String> tm1 = new TreeMap<>();
    }
}

```

```

Student s1 = new Student("张三", 23);
Student s2 = new Student("李四", 25);
Student s3 = new Student("王五", 24);
tm1.put(s1, "基础班");tm1.put(s2, "基础班");tm1.put(s3, "就业班");
System.out.println(tm1);
//方法 2: 定制排序, 通过 Comparator 升序排序,
/*TreeMap<Teacher,String> tm2 = new TreeMap<>();
Teacher t1 = new Teacher("张三",23);
Teacher t2 = new Teacher("李四",25);
Teacher t3 = new Teacher("王五",24);
tm2.put(t1,"基础班");
tm2.put(t2,"基础班");
tm2.put(t3,"就业班");
System.out.println(tm2);*/
TreeMap<Teacher, String> tm2 = new TreeMap<>(new Comparator<Teacher>() {
    @Override
    public int compare(Teacher t1, Teacher t2) {
        return t1.getAge() - t2.getAge();
    }
});
Teacher t1 = new Teacher("张三", 23);
Teacher t2 = new Teacher("李四", 25);
Teacher t3 = new Teacher("王五", 24);
tm2.put(t1, "基础班");
tm2.put(t2, "基础班");
tm2.put(t3, "就业班");
//System.out.println(tm2);
Set keySet = tm2.keySet();
Iterator it = keySet.iterator();
while (it.hasNext()) {
    Object key = it.next();
    Object value = tm2.get(key); // 获取每个键所对应的值
    System.out.println(key + "--" + value);
}
}
}

```

运行结果:

```

D:\Develop\Java\jdk-11\bin\java.exe "-javaagent:D:\Develop\IntelliJ IDEA
{hm001=王五, hm002=张三, hm003=李四, hm004=赵六}
{name=李四, age=25=基础班, name=王五, age=24=就业班, name=张三, age=23=基础班}
name=张三, age=23--基础班
name=王五, age=24--就业班
name=李四, age=25--基础班

```

图 3.14 运行结果示意图

13、编写程序，演示 Collections 类中常用的方法。

程序：

```
package com.company.com13;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
/**
 * Collections 集合工具类：
 * static <T> boolean addAll(Collection<? super T> c, T...elements) 将所有指定元素添加到指定集合 c 中
 * static void reverse(List list) 反转指定 List 集合中元素的顺序
 * static void shuffle(List list) 对 List 集合中的元素进行随机排序
 * static void sort(List list) 根据元素的自然顺序对 List 集合中的元素进行排序
 * static void swap(List list,int i,int j) 将指定 List 集合中角标 i 处元素和 j 处元素进行交换
 * 使用二分法搜索指定对象在 List 集合中的索引，查找的 List 集合中的元素必须是有序的
 * static int binarySearch(List list,Object key)
 * static Object max(Collection col) 根据元素的自然顺序，返回给定集合中最大的元素
 * static Object min(Collection col) 根据元素的自然顺序，返回给定集合中最小的元素
 * 用一个新值 newVal 替换 List 集合中所有的旧值 oldVal
 * static boolean replaceAll(List list,Object oldVal,Object newVal)
 */
public class CollectionsDemo{
    public static void main(String[] args) {
        ArrayList<String> list =new ArrayList<>();
        // static <T> boolean addAll(Collection<? super T> c, T...elements)
        // 将所有指定元素添加到指定集合 c 中
        Collections.addAll(list,"a","b","c","d");
        System.out.println(list);
        //static void reverse(List list)反转指定 List 集合中元素的顺序
        Collections.reverse(list);
        System.out.println(list);
        //static void shuffle(List list)对 List 集合中的元素进行随机排序
        Collections.shuffle(list);
        System.out.println(list);
        //sort(List list)根据元素的自然顺序对 List 集合中的元素进行排序
        Collections.sort(list);
        System.out.println(list);
        //swap(List list,int i,int j) 将指定 List 集合中角标 i 处元素和 j 处元素进行交换
        //Collections.swap(list,2,3);
        //System.out.println(list);
        //使用二分法搜索指定对象在 List 集合中的索引，查找的 List 集合中的元素必须是有序的
    }
}
```

```

        //static int binarySearch(List list, Object key)
        int index = Collections.binarySearch(list, "c");
        System.out.println(index);
        //static Object max(Collection col) 根据元素的自然顺序，返回给定集合中最大的元素
        String max = Collections.max(list);
        System.out.println(max);
        //static Object min(Collection col) 根据元素的自然顺序，返回给定集合中最小的元素
        String min = Collections.min(list);
        System.out.println(min);
        //用一个新值 newVal 替换 List 集合中所有的旧值 oldVal
        //static boolean replaceAll(List list, Object oldVal, Object newVal)
        Collections.replaceAll(list, "b", "f");
        System.out.println(list);
    }
}

```

运行结果:

```

D:\Develop\Java
[a, b, c, d]
[d, c, b, a]
[c, b, d, a]
[a, b, c, d]
2
d
a
[a, f, c, d]

```

图 3.15 运行结果示意图

14、编写程序，演示 Arrays 工具类中常用的方法。

程序:

```

package com.company.com14;
import java.util.Arrays;
/**
 * Arrays 数组工具类:
 * void sort(); 数组排序;
 * String toString(); 打印数组
 * int binarySearch(Object[] a, Object key) 用二分查找法，查找元素，返回索引位置
 * int[] copyOfRange(int[] original, int from, int to) 复制数组元素到一个新数组中
 * void fill(Object[] a, Object val) 传入的元素替换数组中所有的元素
 */
public class ArraysDemo {

```

```

public static void main(String[] args) {
    int[] arr = {10, 50, 30, 20, 40, 60};
    //void sort(); 数组排序
    Arrays.sort(arr);
    //String toString(); 打印数组
    System.out.println(Arrays.toString(arr));
    // int binarySearch(Object[] a, Object key)
    // 使用二分查找法, 查找元素, 返回索引位置
    int index = Arrays.binarySearch(arr, 50);
    System.out.println(index);
    //int[] copyOfRange(int[] original, int from, int to)
    // 复制数组元素到一个新数组中
    int[] arr2 = Arrays.copyOfRange(arr, 1, 3); // 不包含结束索引
    System.out.println(Arrays.toString(arr2));
    //void fill(Object[] a, Object val) 传入的元素替换数组中所有的元素
    Arrays.fill(arr2, 88);
    System.out.println(Arrays.toString(arr2));
}
}

```

运行结果:

```

D:\Develop\Java\jdk-11\bin
[10, 20, 30, 40, 50, 60]
4
[20, 30]
[88, 88]

```

图 3.16 运行结果示意图

15、编写程序, 演示 Lambda 表达式的使用。

程序 1:

```

package com.company.com15;
/**
 * Lambda 表达式: 简化代码书写
 * 1. 使用前提
 * 必须是接口。接口中只能有一个抽象方法, 对使用接口中方法的一种优化。
 * 2. 使用格式
 * (数据类型 参数名 1, 数据类型 参数名 2----) -> {方法体 }
 * () : 代表的是要执行接口中对应的那个重写方法。如果方法中有参数, 那么就需要传递参数。
 * -> : 固定格式。作用是将小括号中的参数传递给后面的大括号方法体中。
 * {} : 代表的是实现接口中那个抽象方法后的方法体。
 * 匿名内部类: 没有名字的特殊局部内部类
 * 格式:

```



```

*      new 类名 或 接口名 () {
*          重写方法
*      }
*  使用场景：作为方法的参数传递
*  传统实现方式：
*      1. 编写实现类
*      2. 重写抽象方法
*      3. 创建实现类对象
*      4. 将实现类对象作为方法的参数传递
*/
public class LambdaDemo01 {
    public static void main(String[] args) {
        useInter(new Inter() {
            @Override
            public void show(String str) {
                System.out.println(str);
            }
        });
        System.out.println("-----");
        useInter(str->System.out.println(str));
    }
    //使用 Inter 接口的方法
    public static void useInter(Inter i){
        i.show("hello");
    }
}
interface Inter{
    public abstract void show(String str);
}

```

运行结果 1:

```

D:\Develop\Ja
hello
-----
hello

```

图 3.17 运行结果示意图

程序 2:

```

package com.company.com15;
import java.util.Comparator;
import java.util.TreeSet;
public class LambdaDemo02 {
    public static void main(String[] args) {
        /*TreeSet<Integer> ts = new TreeSet<>(new Comparator<Integer>() {

```

```

//ALT+回车用 Lambda 替换
@Override
public int compare(Integer o1, Integer o2) {
    return o2-o1;
}
});*/
TreeSet<Integer> ts = new TreeSet<>((o1, o2) -> o2-o1);
//多个参数，数据类型可以不写，如(Integer o1, Integer o2) ->{return o2-o1;}
//可以简化写法
ts.add(5);
ts.add(2);
ts.add(3);
ts.add(1);
ts.add(4);
System.out.println(ts);
}
}

```

运行结果 2:

```

D:\Develop\Java\
[5, 4, 3, 2, 1]

```

图 3.18 运行结果示意图

三、上机体会

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 图形用户界面开发

实验时间 2025 年 11 月 14、21、28 日第 3、4 节

年级 2024 专业班级 计算机科学与技术 1 班

姓名 网球拍 学 号 20050090

评分：

教师签名： 王庆平

上机四 图形用户界面开发

一、上机目的

- 1、了解 GUI 的概念
- 2、掌握 AWT 包中主要类的层次结构
- 3、掌握常用的容器种类
- 4、掌握布局管理器的种类
- 5、掌握事件处理的机制及适配器类
- 6、了解 Swing 包中主要类的层次结构。
- 7、掌握 Swing 常用的容器。
- 8、掌握 Swing 中组件的使用方法。

二、上机任务

1、窗体和面板。

定义一个公共类 FramePanel 继承 Frame，在 main() 方法中创建 FramePanel 对象 frame，然后设置 frame 为可见及其大小，并取消布局管理器，最后创建 Panel 类对象 pan，设置 pan 的大小和背景颜色，并将其添加到 frame 中。

程序 1：（在主函数中直接创建框架对象的方式）

```
package com.company01;

/*
 * 在主函数中直接创建框架对象的方式
 * 优点：简洁直接，无需额外定义类，代码量少；
 * 缺点：主函数臃肿，所有逻辑（配置、组件、事件）堆叠。
 * 适用场景：1）小型 Demo、快速原型验证；
 *             2）仅需显示简单窗口，无复杂功能；
 *             3）新手学习框架 API 的基础使用。
 */
import java.awt.*;

public class FramePanelTest {
    public static void main(String[] args) {
        // 创建 Frame 类对象 frame，设置标题为“TextEditor”
        Frame frame = new Frame("TextEditor");
        frame.setVisible(true);    // 调用 setVisible() 方法，设置窗体可见
        frame.setSize(240, 150);   // 调用 setSize() 方法，设置窗体大小
        frame.setLayout(null);     // 调用 setLayout() 方法，取消布局管理器
        Panel pan = new Panel();   // 创建 Panel 类对象 pan
        pan.setSize(100, 100);     // 调用 setSize() 方法，设置面板大小
        // 调用 setBackground() 方法，设置面板背景颜色为红色
        pan.setBackground(Color.red);
        frame.add(pan);            // 调用 add() 方法把面板添加到窗体中
    }
}
```

程序 2：（使用创建框架子类的方式）

```
package com.company01;

/*
 * 优点：结构清晰，初始化、组件、事件分离封装；
 * 缺点：需额外定义子类，少量增加代码量。
 * 框架子类方式：1）中大型应用程序；
 *                  2）需要自定义窗口功能（如绘图、事件处理）；
 *                  3）多个地方复用相同窗口；
 *                  4）团队协作开发；
 *                  5）追求代码可维护性和扩展性。
 */

import java.awt.*;

// 定义 FramePanel 类，继承 Frame 类
public class FramePanel extends Frame {
    // 定义有参构造方法，传入标题
    public FramePanel(String title) {
        super(title);    // 调用父类的构造方法
        setVisible(true); // 调用 setVisible() 方法，设置窗体可见
        setSize(240, 150); // 调用 setSize() 方法，设置窗体大小
        setLayout(null); // 调用 setLayout() 方法，取消布局管理器
        // setBackground(Color.blue); // 窗体背景颜色为蓝色
    }
}

%-----%

package com.company01;
import java.awt.*;
public class FramePanelTest {
    public static void main(String[] args) {
        // 创建 FramePanel 类对象 frame，设置标题为“TextEditor”
        FramePanel frame = new FramePanel("TextEditor");
        Panel pan = new Panel(); // 创建 Panel 类对象 pan
        pan.setSize(100, 100); // 调用 setSize() 方法，设置面板大小
        // 调用 setBackground() 方法，设置面板背景颜色为红色
        pan.setBackground(Color.red);
        frame.add(pan); // 调用 add() 方法把面板添加到窗体中
    }
}
```

运行结果：

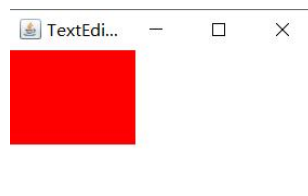


图 4.1 运行结果示意图

2、编写程序， 创建一个用户登录界面，实现输入用户名和密码（设置回显字符为“*”）后，单击“确定”按钮输出用户名和密码。 [注意设置参数](#)：-Dfile.encoding=gbk。

程序：

```
package com.company02;
import java.awt.*;
import java.awt.event.*;
public class UserLogin extends Frame implements ActionListener{
    // 创建用户名标签 lName，并设置标签文本为“用户名”
    Label lName = new Label("用户名");
    // 创建用户名文本框 tName
    TextField tName = new TextField();
    // 创建密码标签 lPass，并设置标签文本为“密码”
    Label lPass = new Label("密码");
    // 创建密码文本框 tPass
    TextField tPass = new TextField();
    // 创建确定按钮 btnConfirm，并设置标签文本为“确定”
    Button btnConfirm = new Button("Enter");
    public UserLogin(String title) {
        // 调用父类构造方法，设置窗体标题
        super(title);
        // 设置窗体大小
        setSize(300, 200);
        // 取消窗体的布局管理器
        setLayout(null);
        // 设置用户名标签与文本框的位置与大小
        lName.setBounds(60, 50, 70, 20);
        tName.setBounds(135, 50, 100, 20);
        // 设置密码标签与文本框的位置与大小
        lPass.setBounds(60, 90, 70, 20);
        tPass.setBounds(135, 90, 100, 20);
        tPass.setEchoChar('*'); // 设置密码文本框的回显字符
        // 设置确定按钮的位置和大小
        btnConfirm.setBounds(110, 140, 70, 20);
        // 为确定按钮注册事件侦听器
        btnConfirm.addActionListener(this);
        // 将组件添加到窗体中
        add(lName);
        add(tName);
        add(lPass);
        add(tPass);
        add(btnConfirm);
        // 设置窗体的位置与可见性
        setLocation(200, 100);
        setVisible(true);
    }
}
```

```

    }
    // 事件处理方法
    public void actionPerformed(ActionEvent e) {
        // 获取事件对象
        Object ob = e.getSource();
        // 如果事件对象为 btnConfirm, 则输出用户名和密码
        if (ob == btnConfirm) {
            System.out.println("用户名: " + tName.getText());
            System.out.println("密码: " + tPass.getText());
        }
    }
}
}
%-----%

public class UserLoginTest {
    public static void main(String[] args) {
        // 创建 UserLogin 类对象
        new UserLogin("登录窗口");
    }
}
}

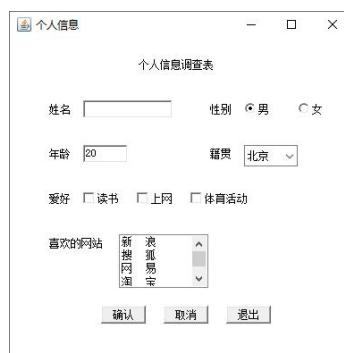
```

运行结果:



图 4.2 运行结果示意图

3、编写程序, 创建一个个人信息调查窗口, 显示姓名、性别、年龄、籍贯、爱好和喜欢的网站等信息, 效果如下图所示。



程序:

```

package com.company03;
import java.awt.*;
import java.awt.event.*;
public class Questionnaire extends Frame implements ActionListener {
    // 创建姓名文本框, 文本框宽度为 10

```

```

TextField name = new TextField(10);
CheckboxGroup sex = new CheckboxGroup();           // 创建性别单选按钮组
Checkbox man = new Checkbox("男", true, sex);      // 创建男单选按钮
Checkbox woman = new Checkbox("女", false, sex);   // 创建女单选按钮
// 创建年龄文本框，初始值为 20，文本框宽度为 4
TextField age = new TextField("20", 4);
Choice nativeplace = new Choice(); // 创建籍贯下拉式菜单
Checkbox like1 = new Checkbox("读书");           // 创建读书复选框
Checkbox like2 = new Checkbox("上网");           // 创建上网复选框
Checkbox like3 = new Checkbox("体育活动"); // 创建体育活动复选框
List website = new List(4); // 创建喜欢的网站列表，显示 4 行
Button btnOK = new Button("确认");               // 创建确认按钮
Button btnCancel = new Button("取消");           // 创建取消按钮
Button btnQuit = new Button("退出");             // 创建退出按钮
// 创建标签
Label label = new Label("个人信息调查表");
Label lName = new Label("姓名");
Label lSex = new Label("性别");
Label lAge = new Label("年龄");
Label lNativeplace = new Label("籍贯");
Label lLike = new Label("爱好");
Label lWebsite = new Label("喜欢的网站");
public Questionnaire(String title) {
    super(title); // 调用父类构造方法
    setSize(400, 400); // 设置窗体的大小
    setLayout(null); // 取消窗体的布局管理器
    // 设置组件的位置和大小
    label.setBounds(150, 50, 100, 20); // 个人信息调查表标签
    lName.setBounds(50, 100, 40, 20); // 姓名标签
    name.setBounds(90, 100, 100, 20); // 姓名文本框
    lSex.setBounds(230, 100, 40, 20); // 性别标签
    man.setBounds(270, 100, 60, 20); // 男单选按钮
    woman.setBounds(330, 100, 60, 20); // 女单选按钮
    lAge.setBounds(50, 150, 40, 20); // 年龄标签
    age.setBounds(90, 150, 50, 20); // 年龄文本框
    lNativeplace.setBounds(230, 150, 40, 20); // 籍贯标签
    nativeplace.setBounds(270, 150, 60, 20); // 籍贯选项框
    // 设置下拉菜单选项，添加城市名
    nativeplace.add("北京");
    nativeplace.add("上海");
    nativeplace.add("天津");
    nativeplace.add("重庆");
    nativeplace.add("广东");
    nativeplace.add("河南");

```



```

// 设置组件的位置和大小
lLike.setBounds(50, 200, 40, 20);      // 爱好标签
like1.setBounds(90, 200, 60, 20);      // 读书标签
like2.setBounds(150, 200, 60, 20);     // 上网标签
like3.setBounds(210, 200, 100, 20);    // 体育活动标签
lWebsite.setBounds(50, 250, 80, 20);   // 喜欢的网站标签
website.setBounds(130, 250, 100, 60);  // 喜欢的网站列表
// 设置列表选项，添加网站名
website.add("新浪");
website.add("搜狐");
website.add("网易");
website.add("淘宝");
website.add("赶集网");
website.add("新华网");
// 设置组件的位置和大小
btnOK.setBounds(110, 330, 50, 20);     // 确认按钮
btnCancel.setBounds(180, 330, 50, 20); // 取消按钮
btnQuit.setBounds(250, 330, 50, 20);   // 退出按钮
// 将各组件添加到窗体中
add(label);
add(lName);
add(name);
add(lSex);
add(man);
add(woman);
add(lAge);
add(age);
add(lNativeplace);
add(nativeplace);
add(lLike);
add(like1);
add(like2);
add(like3);
add(lWebsite);
add(website);
add(btnOK);
add(btnCancel);
add(btnQuit);
setLocationRelativeTo(null); // 使窗体在屏幕上居中放置
setVisible(true);           // 设置窗体可见
// 为三个按钮注册事件侦听器
btnOK.addActionListener(this);
btnCancel.addActionListener(this);
btnQuit.addActionListener(this);

```

```

}
// 重写事件处理方法
public void actionPerformed(ActionEvent e) {
    Object ob = e.getSource(); // 获取事件对象
    if (ob == btnQuit) { // 单击退出按钮
        System.exit(0); // 退出系统
    } else if (ob == btnOK) { // 单击确认按钮
        // 输出信息
        System.out.println("姓名: " + name.getText());
        System.out.println("性别: " + sex.getSelectedCheckbox().getLabel());
        System.out.println("年龄: " + age.getText());
        System.out.println("籍贯: " + nativeplace.getSelectedItem());
        // 如果复选框被选中, 则返回其标签, 否则将字符串设置为空
        String s1 = like1.getState() ? like1.getLabel() + " " : "";
        String s2 = like2.getState() ? like2.getLabel() + " " : "";
        String s3 = like3.getState() ? like3.getLabel() + " " : "";
        System.out.println("爱好: " + s1 + s2 + s3);
        System.out.println("喜欢的网站: " + website.getSelectedItem());
    } else if (ob == btnCancel) { // 单击取消按钮
        name.setText(""); // 清空姓名文本框
        sex.setSelectedCheckbox(man); // 选中“男”单选钮
        age.setText("20"); // 设置年龄文本框为 20
        like1.setState(false); // 取消爱好复选框
        like2.setState(false);
        like3.setState(false);
        // 取消所选喜欢的网站
        website.deselect(website.getSelectedIndex());
    }
}

public static void main(String[] args) {
    new Questionnaire("个人信息");
}
}

```

运行结果:



图 4.3 运行结果示意图

4、编写程序，窗口中放置一个文本区和一个按钮。两个组件都处理鼠标事件，当鼠标在文本区单击时，文本区以红色字体显示鼠标单击的位置坐标；鼠标单击按钮时，文本区以蓝色字体显示鼠标单击的位置坐标，并且窗体能够响应关闭操作。

程序：

```
package com.company04;
import java.awt.*;
import java.awt.event.*;
public class EventAdapter extends Frame {
    private TextArea txtApp = new TextArea("这是文本区");
    private Button btnApp = new Button("这是按钮");
    // 创建鼠标事件侦听器
    MouseEventHandler handler = new MouseEventHandler();
    public EventAdapter(String title) {
        super(title);
        // 为窗体注册窗口事件侦听器
        addWindowListener(new WindowAdapter() {
            // 重写 windowClosing() 方法，关闭当前窗口
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
        // 设置窗体的大小，取消其布局管理器，并使窗体在屏幕上居中放置
        setSize(300, 200);
        setLayout(null);
        setLocationRelativeTo(null);
        // 设置文本区的字体、位置和大小
        txtApp.setFont(new Font("宋体", Font.PLAIN, 16));
        txtApp.setBounds(10, 40, 280, 100);
        // 为文本区注册鼠标事件侦听器
        txtApp.addMouseListener(handler);
        // 设置按钮的位置和大小
        btnApp.setBounds(120, 150, 60, 30);
        // 为按钮注册鼠标事件侦听器
        btnApp.addMouseListener(handler);
        // 将文本区和按钮添加到窗体中
        add(txtApp);
        add(btnApp);
        // 显示窗体
        setVisible(true);
    }
    public static void main(String args[]) {
        new EventAdapter("Adapter Demo");
    }
    // 定义继承 MouseAdapter 适配器的事件侦听器类
```

```

class MouseEventHandler extends MouseAdapter {
    public void mousePressed(MouseEvent e) {
        // 检测事件源是文本区还是按钮
        if (e.getSource() == txtApp) {
            String s = "鼠标所单击的位置为文本区! \n";
            s += "鼠标所处的位置为:\nX=" + e.getX() + ";Y=" + e.getY();
            // 用红色字体显示鼠标单击的位置坐标
            txtApp.setForeground(Color.RED);
            txtApp.setText(s);
        }
        if (e.getSource() == btnApp) {
            String s = "鼠标所单击的位置为按钮! \n";
            s += "鼠标所处的位置为:\nX=" + e.getX() + ";Y=" + e.getY();
            // 用蓝色字体显示鼠标单击的位置坐标
            txtApp.setForeground(Color.BLUE);
            txtApp.setText(s);
        }
    }
}

```

运行结果:

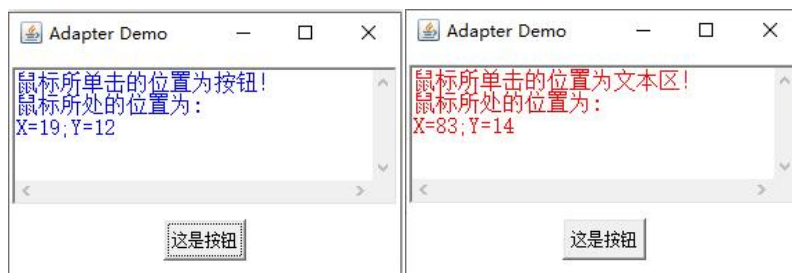


图 4.4 运行结果示意图

5、创建学生信息管理系统，实现添加、删除和退出功能。单击“添加”按钮将信息添加到表格中并显示，信息包括学号、姓名、性别和出生年份；单击“删除”按钮可删除表格中的选择项；单击“退出”按钮关闭系统。

学号	姓名	性别	出生年份
----	----	----	------

程序：

```
package com.company05;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
public class StudentManager extends JFrame implements ActionListener {
    JTextField tNo = new JTextField(10);    // 创建学号文本框 tNo
    JTextField tName = new JTextField(10);  // 创建姓名文本框 tName
    // 创建性别单选按钮组 groupSex，包含两个单选按钮
    ButtonGroup groupSex = new ButtonGroup();
    JRadioButton man = new JRadioButton("男", true);
    JRadioButton woman = new JRadioButton("女");
    // 创建出生年份下拉列表框 cbAge
    JComboBox<String> cbAge = new JComboBox<String>();
    JButton btnOK = new JButton("添加");    // 创建添加按钮 btnOK
    JButton btnDelete = new JButton("删除"); // 创建删除按钮 btnDelete
    JButton btnQuit = new JButton("退出");  // 创建退出按钮 btnQuit
    JLabel lNo = new JLabel("学号");        // 创建学号标签 lNo
    JLabel lName = new JLabel("姓名");      // 创建姓名标签 lName
    JLabel lSex = new JLabel("性别");       // 创建性别标签 lSex
    JLabel lAge = new JLabel("出生年份");   // 创建出生年份标签 lAge
    JTable table;                          // 声明表格 table
    DefaultTableModel model;               // 创建 DefaultTableModel 类对象 model
    public StudentManager(String title) {
        super(title);                      // 调用父类构造方法
        setSize(400, 400);                 // 设置窗体的大小
        // 设置组件位置和大小
        lNo.setBounds(50, 20, 100, 20);
        tNo.setBounds(150, 20, 100, 20);
        lName.setBounds(50, 50, 100, 20);
        tName.setBounds(150, 50, 100, 20);
        lSex.setBounds(50, 80, 100, 20);
        man.setBounds(150, 80, 60, 20);
        woman.setBounds(250, 80, 60, 20);
        lAge.setBounds(50, 110, 100, 20);
        cbAge.setBounds(150, 110, 100, 20);
        // 向 groupSex 组添加单选按钮 man 和 woman
        groupSex.add(man);
        groupSex.add(woman);
        // 设置下拉列表框选项，添加年份
        cbAge.addItem("1990");
        cbAge.addItem("1989");
        cbAge.addItem("1988");
    }
}
```

```

        cbAge.addItem("1987");
        cbAge.addItem("1986");
        cbAge.addItem("1985");
        cbAge.addItem("1984");
        // 设置 3 个按钮位置和大小
        btnOK.setBounds(80, 160, 60, 20);
        btnDelete.setBounds(150, 160, 60, 20);
        btnQuit.setBounds(220, 160, 60, 20);
        Container c = getContentPane(); // 创建 Container 类对象 c
        JPanel panel=new JPanel();      // 创建面板对象 panel
        panel.setLayout(null);           // 取消 panel 的布局管理器
        // 将各组件添加到面板 panel 中
        panel.add(tNo);
        panel.add(tName);
        panel.add(man);
        panel.add(woman);
        panel.add(cbAge);
        panel.add(lNo);
        panel.add(lName);
        panel.add(lSex);
        panel.add(lAge);
        panel.add(btnOK);
        panel.add(btnDelete);
        panel.add(btnQuit);
        String[] colName = { "学号", "姓名", "性别", "出生年份" };
        // 创建 model
        model = new DefaultTableModel(colName, 0);
        // 创建 table, 并使用 model 保存 table 数据
        table = new JTable(model);
        // 创建滚动面板 scrollPane, 将 table 添加到 scrollPane 中
        JScrollPane scrollPane = new JScrollPane(table);
        // 设置窗体的布局管理器为 GridLayout, 分为 2 行 1 列
        c.setLayout(new GridLayout(2, 1));
        c.add(panel);                // 将 panel 添加到窗体中
        c.add(scrollPane);           // 将 scrollPane 添加到窗体中
        setLocationRelativeTo(null); // 使窗体在屏幕上居中放置
        setVisible(true);           // 设置窗体可见
        // 为三个按钮注册事件侦听器
        btnOK.addActionListener(this);
        btnDelete.addActionListener(this);
        btnQuit.addActionListener(this);
    }
    // 重写 ActionEvent 事件处理方法
    public void actionPerformed(ActionEvent e) {

```

```

Object ob = e.getSource(); // 获取事件对象
if (ob == btnQuit) {       // 单击退出按钮
    System.exit(0);         // 退出系统
} else if (ob == btnOK) {   // 单击确认按钮
    // 根据选择获取性别
    String sex;
    if (man.isSelected())
        sex = "男";
    else
        sex = "女";
    String[] stuInfo = { tNo.getText(), tName.getText(),
        sex, cbAge.getSelectedItem().toString() };
    model.addRow(stuInfo); // 将信息添加到 model 中
    tNo.setText("");        // 清空学号文本框
    tName.setText("");      // 清空姓名文本框
    man.setSelected(true);  // 选中 man 单选按钮
} else if (ob == btnDelete) { // 单击删除按钮
    if (table.getSelectedRow() < 0)
        // 弹出一个错误提示对话框
        JOptionPane.showMessageDialog(null, "请在表格中选择要删除的选项",
            "警告", JOptionPane.WARNING_MESSAGE);
    else
        // 删除选择项
        model.removeRow(table.getSelectedRow());
}
}

public static void main(String[] args) {
    new StudentManager("学生信息管理");
}
}

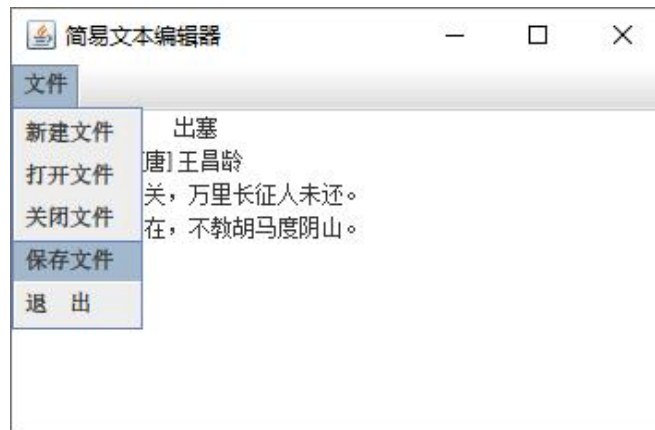
```

运行结果：

学号	姓名	性别	出生年份
001	张三	男	1990
002	李四	女	1989

图 4.5 运行结果示意图

6、创建一个简易文本编辑器，包括一个主菜单和一个文本区。其中，主菜单包括“新建文件”“打开文件”“关闭文件”“保存文件”和“退出”选项。



程序：

```
package com.company06;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import javax.swing.*;

public class TextEditor extends JFrame implements ActionListener {
    JMenuBar mainMenubar; // 声明菜单栏
    JMenu file; // 声明主菜单项
    JMenuItem nw; // 声明新建文件子菜单项
    JMenuItem op; // 声明打开文件子菜单项
    JMenuItem cl; // 声明关闭文件子菜单项
    JMenuItem sf; // 声明保存文件子菜单项
    JMenuItem ex; // 声明退出子菜单项
    JTextArea tx; // 声明文本区

    public TextEditor(String title) {
        super(title); // 调用父类构造方法
        setSize(400, 400); // 设置窗体大小
        setLocationRelativeTo(null); // 使窗体在屏幕上居中放置
        menuinit(); // 调用构建与处理菜单的方法
        tx = new JTextArea(); // 创建文本区对象
        add(tx); // 将文本区对象放入窗体
        setVisible(true); // 使窗体可见
    }

    // 定义菜单构建与处理方法
    void menuinit() {
        mainMenubar = new JMenuBar(); // 创建主菜单栏
        file = new JMenu("文件"); // 创建主菜单项
        // 创建各子菜单项
        nw = new JMenuItem("新建文件");
        op = new JMenuItem("打开文件");
```



```

        cl = new JMenuItem("关闭文件");
        sf = new JMenuItem("保存文件");
        ex = new JMenuItem("退出");
        // 将各子菜单项加入到主菜单项中
        file.add(nw);
        file.add(op);
        file.add(cl);
        file.add(sf);
        file.add(ex);
        mainMenubar.add(file);    // 将主菜单项加入到主菜单栏
        setJMenuBar(mainMenubar); // 为窗体设置主菜单
        // 为各子菜单项注册事件侦听器
        nw.addActionListener(this);
        op.addActionListener(this);
        cl.addActionListener(this);
        sf.addActionListener(this);
        ex.addActionListener(this);
    }
    // 重写(ActionEvent)事件处理方法
    public void actionPerformed(ActionEvent e) {
        Object ob = e.getSource();    // 获取事件对象
        JFileChooser f = new JFileChooser(); // 创建文件选择器对象
        // 选择"新建文件"或"关闭文件"子菜单项
        if ((ob == nw) || (ob == cl)) {
            tx.setText("");    // 清空文本区
        } else if (ob == op) {    // 选择"打开文件"子菜单项
            // 弹出具有自定义按钮的文件选择器对话框
            f.showOpenDialog(this);
            try {
                // 定义StringBuffer对象s
                StringBuffer s = new StringBuffer();
                // 创建FileReader对象in, 参数为在文件选择器中选中的文件
                FileReader in = new FileReader(f.getSelectedFile());
                // 读取文件内容, 将其追加到s中
                while (true) {
                    int b = in.read();
                    if (b == -1)
                        break;
                    s.append((char) b);
                }
                tx.setText(s.toString()); // 将文件内容显示在文本区
                in.close();    // 关闭文件
            } catch (Exception e1) {
            }
        }
    }

```

```

    } else if (ob == sf) {          // 选择“保存文件”子菜单项
        f.showSaveDialog(this);    // 显示文件选择对话框
        try {
            // 创建 FileWriter 对象，其参数为选择的文件
            FileWriter out = new FileWriter(f.getSelectedFile());
            out.write(tx.getText()); // 将文本区内容写入文件
            out.close();             // 关闭文件
        } catch (Exception e2) {
        }
    } else if (ob == ex)          // 选择“退出”子菜单项
        System.exit(0);          // 退出系统
    }

    public static void main(String[] args) {
        new TextEditor("简易文本编辑器");
    }
}

```

运行结果:



图 4.6 运行结果示意图

7. 利用 BoxLayout 布局，设计登录界面。

程序:

```

package cn.edu.kmust;
import java.awt.*;
import javax.swing.*;
class BoxLayoutDemo extends JFrame {
    private JLabel jLabel1, jLabel2, jLabel3;
    private JButton jConnect;
    private JTextField jUID;
    private JPasswordField jPwd, jConfirmPwd;
    BoxLayoutDemo() {
        super("用户注册界面");
        jLabel1 = new JLabel("用户名:");
        jLabel2 = new JLabel("密码:");
        jLabel3 = new JLabel("确认密码:");
        jConnect = new JButton("注册");
    }
}

```

```

jUID = new JTextField(15);
jPwd = new JPasswordField(15);
jConfirmPwd = new JPasswordField(15);
Box userName = Box.createHorizontalBox();    //创建 Box 容器的水平格式
Box password = Box.createHorizontalBox();
Box confirmPassword = Box.createHorizontalBox();
Box submitButton = Box.createHorizontalBox();
userName.add(jLabel1);                      //在 Box 容器中加入组件
userName.add(jUID);
password.add(jLabel2);
password.add(jPwd);
confirmPassword.add(jLabel3);
confirmPassword.add(jConfirmPwd);
submitButton.add(jConnect);
this.setLayout(new BoxLayout(this.getContentPane(), BoxLayout.Y_AXIS));
//设置 BoxLayout 布局
this.add(userName);
this.add(password);
this.add(confirmPassword);
this.add(submitButton);
userName.setVisible(true);
password.setVisible(true);
confirmPassword.setVisible(true);
submitButton.setVisible(true);
this.setSize(240, 150);
this.setVisible(true);
this.setDefaultCloseOperation(EXIT_ON_CLOSE);
}
public static void main(String[] args) {
    new BoxLayoutDemo();
}
}

```

运行结果:



图 4.7 运行结果示意图

三、上机体会

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 多线程

实验时间 2025 年 12 月 5、12 日 第 3、4 节

年级 2024 专业班级 计算机科学与技术 1 班

姓名 网球拍 学 号 20050090

评分： _____

教师签名： 王庆平

上机五 多线程

一、上机目的

- 1、了解进程与线程,能够说出进程与线程的区别;
- 2、掌握使用 Thread 类、Runnable 接口和 Callable 接口实现多线程;
- 3、熟悉后台线程的使用,能够理解后台线程用于做什么;
- 4、了解线程的生命周期及状态转换;
- 5、能够说出线程的生命周期的 6 种基本状态以及这 6 种状态的转换方式;
- 6、掌握操作线程的相关方法;
- 7、学会正确使用线程的优先级、休眠、合并、让步和中断操作;
- 8、掌握多线程的同步,能够正确地使多线程同步。

二、上机任务

1、继承 Thread 类创建多线程。

程序:

```
package cn.edu.kmust.wqpl;
/**
 * 继承 Thread 类创建多线程步骤:
 * 1、定义线程子类
 * class <ClassName> extends Thread{           // 继承 Thread 类
 *     public void run() {                       // 重写 run() 方法
 *         ...                                   // 线程执行代码
 *     }
 * }
 * 2、创建与启动线程
 * 创建 Thread 子类的实例,即创建线程子类对象,调用 start() 方法即可。
 */
public class MyThread extends Thread{
    // 线程要执行代码
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("子线程:"+i);
        }
    }
}
%-----%

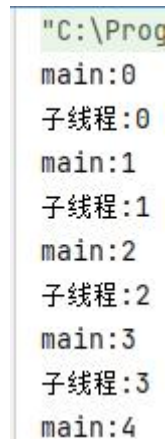
package cn.edu.kmust.wqpl;
public class Test01 {
    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start();
    }
}
```

```

        for (int i = 0; i < 10; i++) {
            System.out.println("main:"+i);
        }
    }
}

```

运行结果:



```

"C:\Prog
main:0
子线程:0
main:1
子线程:1
main:2
子线程:2
main:3
子线程:3
main:4

```

图 6.1 运行结果示意图

2、实现 Runnable 接口创建多线程。

程序:

```

package cn.edu.kmust.wqp2;

/**
 * 实现 Runnable 接口创建多线程的步骤:
 * 1. 定义线程类
 * class <ClassName> implements Runnable {    // 实现 Runnable 接口
 *     public void run() {                    // 重写 run()方法
 *         ...                                // 线程执行代码
 *     }
 * }
 * 2. 创建与启动线程
 * 1) 首先基于定义的线程类创建对象;
 * 2) 然后将该对象作为 Thread 类构造方法的参数;
 * 3) 创建 Thread 类对象, 最后通过 Thread 类对象调用 start()方法启动线程。
 */
public class MyRunnable implements Runnable{
    @Override
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("子线程:"+i);
        }
    }
}
%-----%

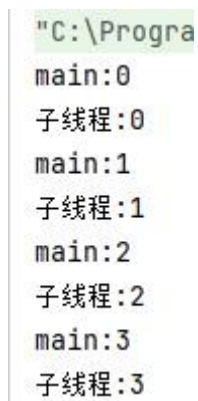
```

```

package cn.edu.kmust.wqp2;
public class Test02 {
    public static void main(String[] args) {
        MyRunnable myRunnable = new MyRunnable();
        Thread thread = new Thread(myRunnable);
        thread.start();
        for (int i = 0; i < 10; i++) {
            System.out.println("main:"+i);
        }
    }
}

```

运行结果:



```

"C:\Progra
main:0
子线程:0
main:1
子线程:1
main:2
子线程:2
main:3
子线程:3

```

图 6.2 运行结果示意图

3、基于 Callable 接口和 FutureTask 类实现多线程。

程序:

```

package cn.edu.kmust.wqp3;
import java.util.concurrent.Callable;
public class MyCallable implements Callable <String>{
    @Override
    public String call() throws Exception {
        for (int i = 0; i < 5; i++) {
            System.out.println("子线程表白第: "+(i+1)+"次");
        }
        return "答应了!!!";
    }
}
%-----%

package cn.edu.kmust.wqp3;
import java.util.concurrent.*;
public class Test03 {
    public static void main(String[] args) throws ExecutionException,
    InterruptedException {
        MyCallable mc = new MyCallable();
        FutureTask<String> ft=new FutureTask<>(mc);

```

```

Thread thread = new Thread(ft); // FutureTask 是 Runnable 实现类
thread.start();
// 用于获取执行结果，该方法会发生阻塞，一直等到任务执行完毕才返回执行结果
String s = ft.get();
System.out.println(s);
    }
}

```

运行结果：

```

"C:\Program Fil
子线程表白第：1次
子线程表白第：2次
子线程表白第：3次
子线程表白第：4次
子线程表白第：5次
答应了!!!

```

图 6.3 运行结果示意图

4、

程序：

%-----%

运行结果：

5、

程序：

%-----%

运行结果：

三、上机体会